

A Unified Dual-Loop Architecture for Mathematical Discovery and Lean-Verified Theorem Proving

**OmegaClaw-Math, PeTTa/MORK, LLM and Hyperon research agents,
GNN/PLN proof search, Pantograph, and Lean 4**

Unified design, version 3.0 - 16 July 2026

Contents

Section	Subject
Extended Abstract	The combined architecture and its central commitments
1	From two ingredient architectures to one proof system
2	Design goals, non-goals, and governing invariants
3	End-to-end architecture and authority boundaries
4	The dual-loop proof process
5	The unified MORK proof metagraph
6	The Lean-native formal ATP service
7	The bidirectional lemma and obstruction bridge
8	Hierarchical scheduling, leasing, and parallel search
9	Context compilation and proof memory
10	Autonomous proof cycles and escalation policy
11	Epistemic status, formal soundness, and verification
12	OmegaClaw, ThreadKeeper, Hyperon, and skill architecture
13	Reproducible computation, artifacts, models, and environments
14	Operating profiles and worked flows
15	Migration, deployment, and compatibility
16	Evaluation methodology and implementation roadmap
Appendix A	Unified MORK atom reference
Appendix B	Formal ATP and worker contracts
Appendix C	Prompt and skill specifications
Appendix D	Configuration example
References	Ingredient systems and research background

Document status. Version 3.0 is a new combined architecture and implementation blueprint. It takes the earlier MORK-native OmegaClaw-Math proof architecture and the current Mathematics-ai-atp project as ingredient designs, but the system specified here is a larger architecture with new boundaries, schemas, control loops, and soundness rules. Repository-specific observations reflect public source material reviewed on 16 July 2026. The companion code is a runnable protocol scaffold, not a claim that the full integrated system already exists in either upstream repository.

Extended Abstract

Mathematical AI has at least two very different time scales. At the short scale, a prover is given an exact formal goal, predicts a tactic and its arguments, applies the tactic in a proof assistant, receives a list of new proof states, and continues until the goal is discharged or the search budget is exhausted. At the long scale, a research system must choose what theorem to attack, invent intermediate lemmas, compare several strategies, recognize that a formulation is wrong or awkward, run experiments, preserve failed ideas, move between informal and formal representations, and resume coherent work after days or months. A robust mathematical agent needs both scales. Treating either one as a thin add-on to the other loses the distinctive strengths of the missing scale.

The Mathematics-ai-atp repository supplies a concrete formal-search ingredient. Its public design describes GNN-based tactic and argument prediction, PLN-style reasoning and ranking, an AND/OR proof hypergraph, and tactic execution through Lean and Pantograph.¹ Its hypergraph source treats alternative tactics as OR choices and the subgoals generated by one tactic as an AND set of obligations; the current implementation uses a tree of AND-hyperedges and explicitly identifies shared multi-parent transpositions as future work.² This is the right basic topology for local proof search. It is naturally anchored in exact Lean states, and it gains a strong correctness boundary from the Lean kernel.

The earlier OmegaClaw-Math architecture supplies a different ingredient: a persistent proof project represented in a MORK-backed PeTTa metagraph, a revisioned event journal, a content-addressed artifact store, an OmegaClaw coordinator, bounded ThreadKeeper workers, Hyperon/PLN creativity, context compilation, critics, experiments, and incremental Lean formalization. Its central insight remains valid: **the persistent proof project, not an LLM conversation, is the agent.** A model call is a replaceable worker. The durable mathematical program, its alternatives, its provenance, and its verification state live outside the chat.

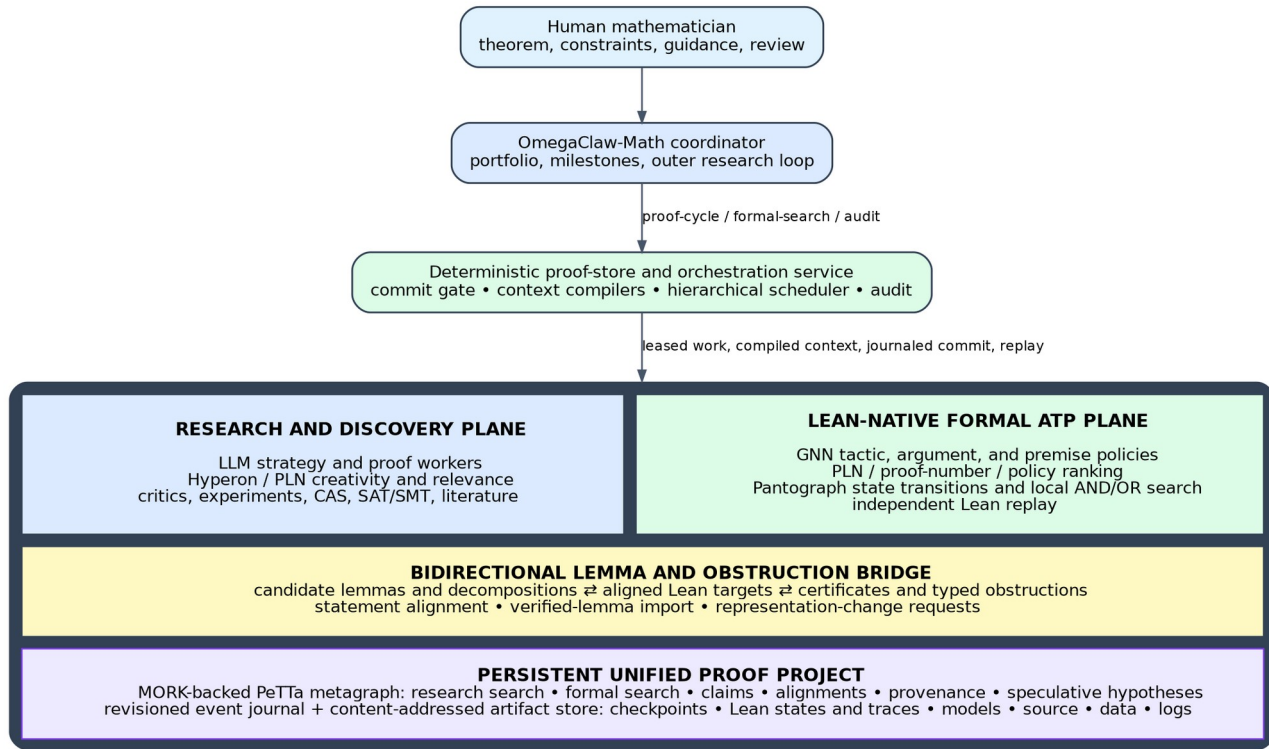
Version 3 unifies these ingredients through **two nested proof-search loops over one persistent typed metagraph.**

1. The **inner formal loop** is a Lean-native ATP service. It receives an exact declaration, a pinned environment, accessible premises, and a budget. It uses learned tactic and premise policies, PLN or other heuristic annotations, Pantograph state transitions, and an AND/OR proof hypergraph to search for a kernel-checkable certificate.
2. The **outer research loop** is managed by OmegaClaw-Math. It chooses which claim or route deserves effort, compiles context, invokes LLM and Hyperon workers, proposes bridge lemmas and representation changes, runs critics and experiments, interprets formal failures, and manages long-horizon strategy.

The loops are neither peers with duplicate authority nor a monolithic flat search. Lean is the authority for exact formal correctness. LLMs and Hyperon are the primary engines for strategic invention, abstraction, analogy, decomposition, and research management. GNN and PLN scores select promising actions but never certify truth. The deterministic proof-store and commit gate mediate every durable state change.

¹ iCog-Labs-Dev, *Mathematics-ai-atp* README, main branch, reviewed 16 July 2026: <https://github.com/iCog-Labs-Dev/Mathematics-ai-atp>

² iCog-Labs-Dev, *maths_ai/hybrid_reasoner/hypergraph.py*, main branch, reviewed 16 July 2026: https://github.com/iCog-Labs-Dev/Mathematics-ai-atp/blob/main/maths_ai/hybrid_reasoner/hypergraph.py



Unified end-to-end architecture

The two loops share one **unified MORK proof metagraph**, but the graph is typed so that different epistemic objects do not collapse together. It contains:

- a research search graph of informal or semiformal states and moves;
- a formal proof-search graph of exact Lean states and tactic applications;
- a statement-alignment layer connecting mathematical claims to Lean declarations;
- a justification graph of claims, dependencies, criticism, and formal certificates;
- a provenance graph of attempts, model versions, context packets, artifacts, environments, formal runs, and replays;
- a speculative layer for Hyperon and LLM hypotheses;
- derived views for frontiers, routes, transpositions, dependency closures, taint, and reusable verified lemmas.

The persistent graph does not require every individual Lean transition to be a heavyweight global transaction. A formal ATP worker maintains a fast local proof-hypergraph cache and emits durable checkpoints and trace chunks. The MORK metagraph is the semantic source of truth for the project-level state, while the local cache is a materialized view for one formal run. Exact state hashes and environment manifests permit safe resume and replay. Semantic signatures can propose transpositions, but states are merged only after an exact or formally justified equivalence check.

The architecture introduces an explicit **bidirectional lemma and obstruction bridge**. The outer loop can send a candidate bridge lemma, decomposition, invariant, premise neighborhood, or representation change to the formal ATP service. The formal service can return a verified lemma, a partially closed decomposition, a repeated tactic failure, a minimal unsolved Lean state, a missing-library diagnosis, a typeclass or coercion obstruction, or evidence that the formal statement is stronger than intended. These outputs become structured project events rather than disappearing into console logs.

This bridge is the most important integration point. A local tactic predictor is good at selecting the next move inside a representation. A research agent can change the representation, invent a missing lemma, or restructure the theorem.

Conversely, informal mathematical creativity is safer and more useful when it receives precise formal feedback and can return verified lemmas to the global proof program.

The combined architecture resolves the recurring problem of assigning a single scalar value to a mathematical claim by refusing to do so. A claim has a discrete protocol status and a provenance-bearing evidence record. A search action has a vector of heuristic features such as GNN prior, premise relevance, PLN strength and confidence, proof number, novelty, information gain, dependency centrality, expected verification value, and cost. A scheduler may derive a temporary policy-specific priority from those features. The priority is not the truth value of the claim.

The formal soundness boundary is deliberately strict:

- every subgoal produced by Lean is retained as a required child of the tactic edge;
- heuristic scores cannot mark a formal state solved;
- only an actual zero-goal Lean transition can close a leaf;
- a complete proof claim requires a replayable tactic trace or proof term;
- final promotion requires independent replay in the pinned Lean environment with the expected imports and dependency hashes;
- sorry, undeclared axioms, omitted obligations, and environment drift are rejected or made explicit by policy.

The architecture supports three operating profiles. A **formal-native profile** begins with a Lean declaration and uses the ATP inner loop immediately; the outer loop intervenes only when local search stagnates or a new lemma is needed. A **research-native profile** begins with an informal theorem and uses the outer loop to build a strategy and claim ledger, formalizing stable claims incrementally. A **hybrid co-development profile** lets informal and formal representations evolve together, with verified lemmas continually reshaping global search.

Persistent autonomy remains finite and auditable. A global scheduler leases a research move, formal run, critic task, or experiment. A formal run may execute thousands of local transitions inside its budget and return periodic checkpoints. After repeated formal non-progress, the router can widen premise retrieval, diversify the tactic policy, invoke a missing-lemma procedure, request a representation change, or revisit the statement alignment. Workers never mutate committed proof state directly. The deterministic commit gate validates schemas, theorem identity, statuses, dependencies, all referenced artifacts, lease fencing tokens, and formal replay results.

The companion implementation package extends the version 2 scaffold with formal-state, tactic-application, formal-search, statement-alignment, obstruction, and score-vector schemas; illustrative MORK atoms and MM2 rules; a formal ATP adapter boundary; a deterministic fake ATP used for protocol tests; new OmegaClaw skills and worker prompts; and regression tests for the core soundness invariants. The package is designed so that the existing Mathematics-ai-atp code can first be wrapped as a service, then gradually refactored into the common protocol without blocking experimentation.

The resulting system has a simple conceptual center:

Lean is the epistemic core for formal claims; LLMs and Hyperon are the creative and strategic core; MORK is the shared semantic memory; the proof-store is the transactional authority; and the lemma bridge turns local formal search and long-horizon mathematical research into one learning process.

1. From Two Ingredient Architectures to One Proof System

1.1 The formal-search ingredient

The Mathematics-ai-atp project is oriented around exact Lean theorem statements and programmatic proof-state modification. Its public README identifies four central ingredients: GNN tactic and argument prediction, PLN-style reasoning and ranking, proof hypergraph search, and Lean/Pantograph integration.³ This defines a natural local loop:

³ iCog-Labs-Dev, *Mathematics-ai-atp* README, main branch, reviewed 16 July 2026: <https://github.com/iCog-Labs-Dev/Mathematics-ai-atp>

Lean state

- > tactic and argument prediction
- > Lean tactic execution
- > list of exact subgoals
- > heuristic ranking and frontier update
- > next Lean state

The project also exposes a useful conceptual split. The GNN proposes syntactically plausible local actions and premises. PLN contributes a symbolic relevance or provability heuristic. Lean determines whether an action is legal and what obligations it creates. The hypergraph records alternatives and conjunctions. These pieces belong inside the formal ATP service of the new architecture rather than being stretched into a complete research manager.

1.2 The persistent-research ingredient

The earlier OmegaClaw-Math design was motivated by proof efforts that outlast one conversation. Such efforts contain literal targets, intermediate claims, alternative strategies, partial computations, failed reductions, barriers, stronger substitutes, route-level bypasses, external sources, and human guidance. It placed the symbolic project state in a MORK-backed PeTTa metagraph, with an append-only event journal for durability and a content-addressed artifact store for large data. OmegaClaw coordinated bounded workers rather than treating the chat history as the proof database.

That design is retained, but formal proof is no longer described mainly as a downstream verification phase. Formal ATP is now a first-class service with its own persistent subgraph, scheduler, environment manifests, replay contract, and direct feedback into research strategy.

1.3 Why a flat merger is inadequate

A naive merger might add an LLM worker to the ATP repository or add a tactic-search tool to OmegaClaw. Both are useful prototypes, but neither captures the combined system.

A single flat frontier would mix incomparable actions: apply `simp`, search a library, invent a lemma, run a counterexample program, revise a definition, ask a critic, or change the proof representation. Local Lean transitions occur at a vastly higher frequency than global research moves. They require different state identities, budgets, and success criteria. A flat scheduler would either micromanage every tactic from the outer agent or hide strategically important formal feedback inside an opaque subroutine.

The dual-loop architecture instead gives each scale its own scheduler and state machine, connected by durable checkpoints and a typed bridge.

1.4 What is genuinely shared

The two loops share more than a user interface:

- theorem and claim identities;
- exact dependency and statement-alignment records;
- one provenance and artifact system;
- one event journal and commit gate;
- one graph namespace for routes, claims, formal states, and verified lemmas;
- one concept and mechanism memory for Hyperon;
- one evaluation history showing which strategies and formal policies worked;
- one human-facing status model.

They do not share a single undifferentiated node type, a single scalar value, or a single model context.

1.5 The architectural thesis

The main thesis can be stated as four propositions.

1. **Formal local correctness and global mathematical invention are distinct capabilities.** Neither should impersonate the other.
2. **Both capabilities become stronger when they operate over the same persistent proof project.** Verified lemmas flow outward; precise obstructions flow inward.
3. **Authority must remain asymmetric.** Lean certifies exact formal statements; LLMs, GNNs, and PLN guide search.
4. **Integration should occur through stable protocols and typed graph relations, not through shared conversational state.**

2. Design Goals, Non-Goals, and Governing Invariants

2.1 Functional goals

The initial integrated system should:

1. start from either an informal theorem or an exact Lean declaration;
2. preserve all research and formal attempts under stable IDs;
3. represent both research alternatives and Lean tactic AND/OR structure;
4. resume formal proof search from durable checkpoints;
5. distinguish informal claims, aligned formal declarations, exact Lean states, tactic edges, and certificates;
6. let a GNN/PLN-guided ATP solve straightforward formal goals without an LLM call;
7. escalate difficult formal obstructions to LLM/Hyperon workers with a compact, reproducible context;
8. import Lean-verified lemmas into the global claim ledger and reuse them across routes;
9. preserve exact model, corpus, Lean, mathlib, Pantograph, and source versions for every formal run;
10. allow several research workers and formal workers to share a proof safely through leases and fencing tokens;
11. support reproducible experiments, critics, literature artifacts, and human guidance;
12. expose deterministic audit and export commands.

2.2 Non-goals of the first implementation

The first implementation does not need to:

- train a state-of-the-art tactic model;
- replace the existing Mathematics-ai-atp codebase wholesale;
- store every token of every Lean state directly in MORK;
- make MORK itself the sole durability layer;
- solve statement equivalence automatically in full generality;
- run dozens of asynchronous workers before two- or three-worker safety is established;
- learn one universal value function for all mathematical activity;
- turn PLN strength into theorem status;
- formalize every speculative claim immediately;
- infer that an informal theorem is solved merely because a nearby Lean theorem was checked.

2.3 Core invariants

The package treats the following as non-negotiable protocol invariants.

Invariant 1: exact theorem identity

Every proof project has a normalized theorem kernel. Every formal claim has an exact Lean declaration and environment identity. The system never silently changes quantifiers, definitions, assumptions, domains, constructivity, or the literal target.

Invariant 2: all Lean obligations are preserved

When Lean applies a tactic and returns several goals, the tactic edge has all of them as required children. Ranking may decide which child to expand first; it may not decide which obligations exist.

Invariant 3: heuristic evidence cannot close a formal state

GNN probability, premise score, PLN strength/confidence, Thompson samples, LLM confidence, novelty, and scheduler utility are annotations. A formal state closes only through Lean state transitions and certificate replay.

Invariant 4: formal completion is replay-gated

A formal run may report a candidate proof. The project promotes the corresponding formal claim only after an independent replay under a pinned environment and policy checks for forbidden placeholders or axioms.

Invariant 5: statement alignment is explicit

An informal claim and a Lean declaration may be related by `formalizes`, but they are not identical objects. Alignment status and review provenance are stored explicitly.

Invariant 6: persistent state is authoritative

No LLM conversation, worker memory, local ATP cache, or model provider is the authoritative project state. The committed metagraph, journal, and artifact store are.

Invariant 7: workers propose; the commit gate decides

Research workers, Hyperon, formal ATP services, critics, and experiment processes emit structured proposals and artifacts. Only the deterministic proof-store service mutates committed project state.

Invariant 8: raw records are immutable

Prompts, responses, Lean diagnostics, proof traces, model outputs, experiments, and event bundles are append-only. Summaries and indexes are derived views.

Invariant 9: status and priority are separate

A claim's epistemic status is discrete and provenance-bearing. A scheduler priority is temporary and policy-dependent. Updating a priority does not change truth status.

Invariant 10: late and stale work remains inspectable but cannot silently commit

Lease fencing, base revisions, environment hashes, and theorem hashes prevent obsolete results from changing current state. Their artifacts remain available for audit or explicit rebase.

2.4 Why no single scalar claim value exists

The phrase “value of a mathematical claim” conflates several questions:

- Is the claim true?

- Is it formally verified?
- Is it central to the target theorem?
- Is it easy to test?
- Is it likely to be provable with the current library?
- Would a counterexample be informative?
- Is it novel relative to existing branches?
- How expensive is the next attempt?

The architecture stores a feature record rather than one scalar:

```
protocol_status
formal_status
dependency_centrality
gnn_tactic_prior
premise_relevance
pln_strength
pln_confidence
proof_number
disproof_number
novelty
expected_information_gain
verification_value
estimated_cost
repeated_failure_risk
```

The global scheduler and local ATP scheduler derive different priorities from overlapping subsets of this vector. The values and weights are recorded with each lease so scheduling decisions remain auditable.

3. End-to-End Architecture and Authority Boundaries

3.1 Component overview

The unified architecture contains eight cooperating responsibilities.

Human mathematician. Defines or imports the theorem kernel, resolves semantic ambiguity, supplies high-value guidance, reviews statement alignment, and approves major claims or resource changes. Human guidance is a search event, not formal evidence.

OmegaClaw-Math coordinator. Owns the user relationship, theorem portfolio, milestone communication, and outer proof cycle. It normally invokes high-level operations rather than editing graph atoms.

Deterministic proof-store and orchestration service. Owns revisions, leases, hierarchical scheduling, context compilation, schema validation, commit, replay, taint propagation, audit, and export. It is the sole writer of committed proof atoms.

MORK-backed PeTTa metagraph. Stores typed research and formal search structure, claims, alignments, provenance, speculative hypotheses, indexes, and derived views. PeTTa and MORK provide a natural shared symbolic substrate for OmegaClaw and Hyperon.⁴⁵

Event journal and artifact store. The journal records every committed change and formal checkpoint. The artifact store retains large theorem text, Lean states, tactic traces, model inputs and outputs, source files, corpora manifests, code, data, papers, and logs by content hash.

Research and discovery plane. Bounded LLM workers, Hyperon/PLN creative operators, critics, experimenters, literature tools, and human interaction propose global mathematical moves.

⁴ TrueAGI, PeTTa repository and MORK-space documentation: <https://github.com/trueagi-io/PeTTa>

⁵ TrueAGI, MORK repository and wiki: <https://github.com/trueagi-io/MORK>

Formal ATP plane. A service wrapping the Lean-centered prover uses GNN tactic and premise policies, PLN and proof-search heuristics, Pantograph, Lean, and a local AND/OR proof hypergraph.

Lemma and obstruction bridge. A typed translation layer connects research claims to exact Lean declarations and converts formal outcomes into verified lemmas or research-level obstructions.

3.2 Authority matrix

Component	May read	May propose	May authoritatively write
OmegaClaw coordinator	project status and milestones	high-level actions and human messages	no proof atoms directly
LLM research worker	compiled research context	claims, strategies, lemmas, reformulations	attempt files only
Hyperon/PLN creative worker	selected MORK neighborhoods	speculative hypotheses and scores	speculative proposal channel only
Formal ATP service	exact formal request and referenced artifacts	formal states, tactic edges, checkpoints, certificates, obstructions	run-local cache and result files only
Lean/Pantograph	exact Lean environment	state transitions and diagnostics	no project state
Critic	claim, dependencies, and evidence	verdict and repair	review artifact only
Experiment worker	declared inputs	results and interpretation	attempt artifact directory only
Commit gate	all authoritative records	transaction plan	event journal, committed atoms, canonical sidecars

3.3 Three authorities and one cache

The architecture separates:

MORK metagraph	semantic and query authority
append-only journal	durability and replay authority
artifact store	byte-level authority for large objects
formal local cache	performance-oriented materialized view for one run

The formal local cache is intentionally not an authority. It can be reconstructed from the formal-run request, checkpoints, and trace artifacts. A crash may lose recent uncheckpointed search effort, but it must not change the meaning or status of the proof project.

3.4 Service boundaries

The proof-store exposes a stable project-level API:

```
proof-create(spec_path) -> proof_id
proof-cycle(proof_id) -> cycle_digest
proof-status(proof_id) -> status_digest
proof-lease-next(proof_id, worker_class, ttl) -> lease
proof-commit-result(proof_id, result_path) -> commit_digest
proof-audit(proof_id) -> audit_report
proof-export(proof_id, destination) -> snapshot
```

The formal ATP adapter exposes a narrower service boundary:

```
formal-search-start(request) -> run_id
formal-search-resume(run_id, budget) -> checkpoint_or_result
formal-search-status(run_id) -> formal_run_digest
```

```
formal-search-cancel(run_id) -> cancellation_record
formal-replay(certificate, environment) -> replay_result
```

The proof-store never calls internal GNN or Pantograph modules directly. It calls the adapter. This localizes changes in the ATP codebase and allows alternative formal provers to implement the same contract.

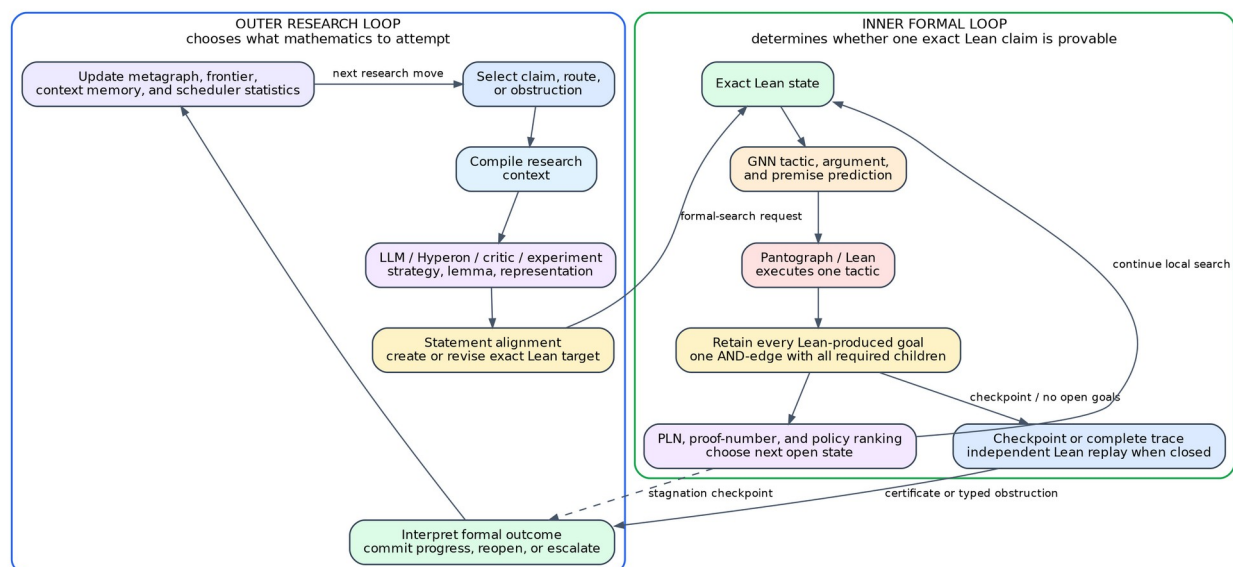
3.5 One proof-store owner per active theorem

The simplest initial concurrency model remains one proof-store process per theorem project. It owns the proof's MORK space, event revision, leases, artifact manifests, and maintenance rules. Several formal and research workers can operate concurrently through the service, but none writes directly to the MORK handle.

A separate global knowledge space may contain reusable concepts, mechanisms, and verified lemmas. Importing one of those objects into a proof is an explicit, versioned event.

4. The Dual-Loop Proof Process

4.1 Two loops with different units of work



Nested outer research loop and inner formal loop

The outer loop operates on **research moves**. A move may be “prove this bridge lemma,” “replace the induction invariant,” “formalize the spectral reduction,” “search for a counterexample,” or “compare two representations.” One outer attempt usually invokes one bounded worker or one formal run.

The inner loop operates on **formal actions**. A formal action is a tactic application with arguments under an exact Lean state. One formal run may contain thousands of such actions.

This separation avoids two bad extremes. The OmegaClaw coordinator does not need to receive a message after every simp or rw. The formal ATP service does not get unilateral authority to change the theorem, introduce a new informal mechanism, or decide that a nearby statement is good enough.

4.2 Outer research loop

A reference outer loop is:

1. Select an eligible research move or formal target.
2. Compile a role-specific context at a specific project revision.
3. Dispatch an LLM, Hyperon, critic, experimenter, or formal ATP run.
4. Preserve complete outputs and artifacts.
5. Validate the structured result.
6. Interpret the result in relation to the literal theorem.
7. Commit accepted claims, states, alignments, obstructions, or formal results.
8. Update derived frontiers, taint, and scheduler statistics.
9. Decide whether to continue locally, escalate, backtrack, or report a milestone.

4.3 Inner formal loop

A reference formal loop is:

1. Select an open exact Lean state from the local frontier.
2. Retrieve accessible premises and corpus neighbors.
3. Predict a bounded set of tactics and arguments.
4. Ask Pantograph/Lean to apply each tactic.
5. For every successful application, retain every resulting subgoal.
6. Attach GNN, PLN, proof-number, and cost annotations.
7. Propagate AND/OR status and re-rank the frontier.
8. Emit checkpoints at policy-defined intervals.
9. On apparent root closure, reconstruct and independently replay the proof.
10. Return a certificate or a typed partial/failure result.

Pantograph is used as a machine-to-machine Lean interface, while Lean’s kernel remains the final checker.⁶⁷

4.4 Control transfer between loops

The inner loop returns control to the outer loop when:

- a certificate is verified;
- the node, time, memory, or tactic budget is exhausted;
- the frontier is empty;
- the same failure signature repeats beyond a threshold;
- premise retrieval appears insufficient;
- the formal statement appears misaligned or too strong;
- a missing lemma is repeatedly suggested by the shape of open goals;
- a representation or library mismatch blocks progress;
- human review is required.

The outer loop can respond by extending the formal budget, widening premise retrieval, changing the local search policy, proposing a lemma, formalizing a new decomposition, revising the statement alignment, or choosing a different route.

4.5 Formal-native and research-native entry points

The system does not impose one universal order.

- In a formal-native task, the root research move is simply “solve Lean declaration D under environment E.” The inner loop starts at once.
- In a research-native task, the root has no aligned Lean declaration. The outer loop builds stable definitions, claims, and strategies before or alongside formalization.
- In a hybrid task, some subclaims are Lean-native while the main theorem remains informal.

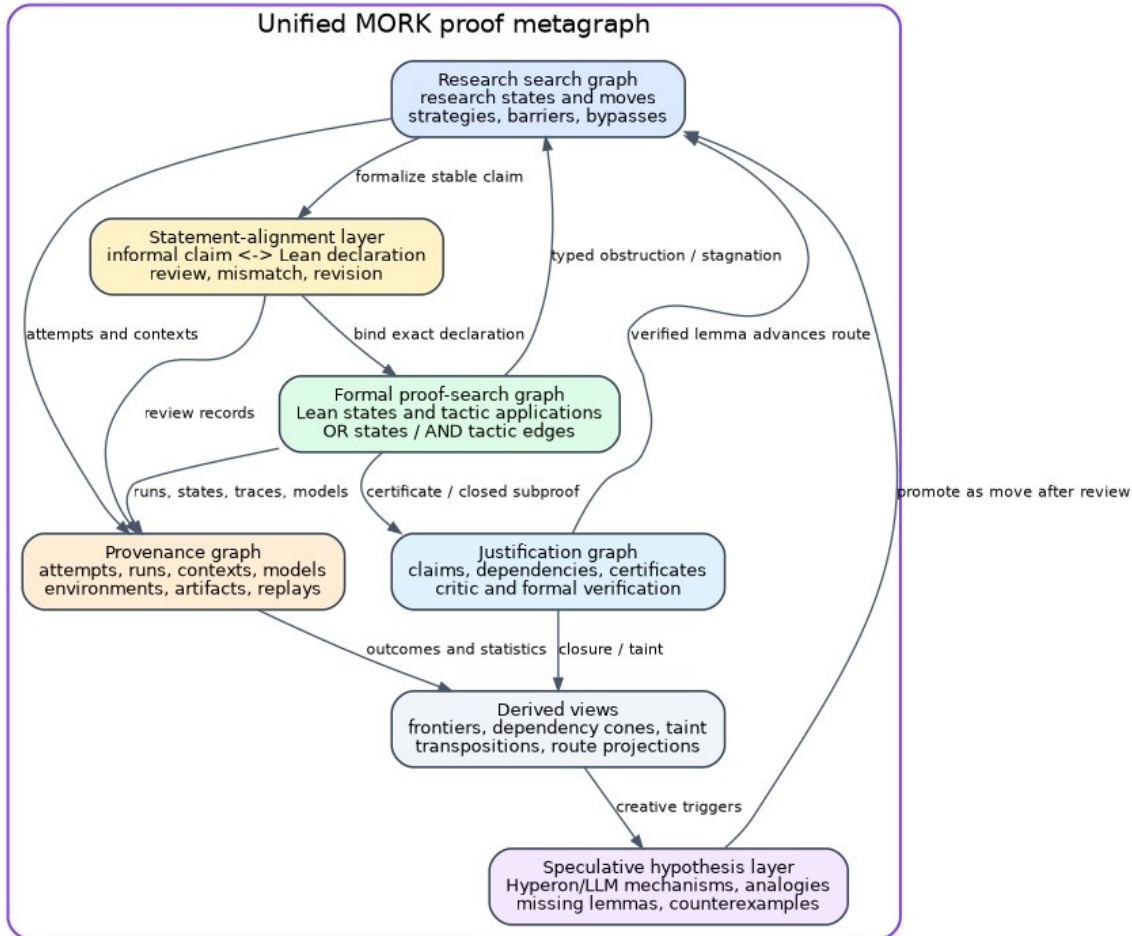
⁶ L. Aniva et al., *Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4*, and PyPantograph: <https://github.com/stanford-centaur/PyPantograph>

⁷ Lean Project, Lean 4 official site and repository: <https://lean-lang.org/> and <https://github.com/leanprover/lean4>

This flexibility is essential. LeanDojo-style benchmarks and speculative research programs have different natural starting points, but they can use the same persistent project architecture.⁸

5. The Unified MORK Proof Metagraph

5.1 Why one metagraph, not one node type



Unified proof metagraph

The metagraph is shared because global and formal reasoning need common identity, provenance, and dependencies. It is typed because informal claims, Lean declarations, exact proof states, tactic edges, hypotheses, and certificates have different semantics.

The graph is best understood as several linked subgraphs rather than one homogeneous database.

5.2 Research search graph

Research states and moves represent the long-horizon program.

- A research state is normally an OR point: one successful route may advance or close the objective.

⁸ K. Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, arXiv:2306.15626, and LeanDojo project resources: <https://arxiv.org/abs/2306.15626>

- A research move may be AND-like if it creates several required claims, but the relation type is explicit because research moves can also bypass, refute, generalize, reformulate, or expose a barrier.
- Distinct routes may converge on the same state or reusable claim.

Core relations include:

```
proposes
requires
alternative
generalizes
specializes
bypasses
supersedes
contradicts
reformulates
exposes-barrier
leaves-open
requests-formal-search
```

5.3 Formal proof-search graph

Formal states are exact Lean obligations. Tactic applications are hyperedges.

- A formal state is an OR node: any replayable outgoing tactic edge may close it.
- A tactic application is an AND edge: every Lean-produced child state must close.
- A childless successful tactic edge represents a real zero-goal Lean transition.
- Tactic rejection is a dead edge with a retained diagnostic.

The persistent representation includes run identity and environment identity. A local formal cache may use integer IDs, but committed IDs are stable and globally namespaced within the proof.

5.4 Statement-alignment layer

An alignment record connects a mathematical claim to a formal declaration:

```
claim c42
formal declaration fd17
alignment al9
```

The alignment stores:

- informal statement artifact and normalized hash;
- Lean source artifact and declaration name;
- imports, namespace, universe and typeclass assumptions;
- quantifier and domain correspondence;
- constructivity and classical assumptions;
- known strengthening or weakening;
- reviewer and review date;
- status: draft, review-needed, aligned, mismatch, superseded, or stale.

A Lean proof promotes the exact formal declaration. The linked informal claim becomes `lean-verified` only when policy accepts the alignment as exact. Otherwise the result is stored as a verified nearby theorem with an explicit relation.

5.5 Justification graph

Claims are reusable mathematical propositions. Dependencies and evidence are explicit:

```
depends-on
supported-by
criticized-by
```

```

proved-by-formal-certificate
contradicted-by
verified-by-exhaustive-computation
imports-verified-claim
invalidates

```

A formal state may be solved without immediately promoting a broad informal claim. Conversely, a research claim may be critic-accepted while not yet formalized.

5.6 Provenance graph

The provenance graph connects:

- research attempts and formal runs;
- exact context manifests;
- prompts and model outputs;
- GNN and premise-model checkpoints;
- corpus and retrieval indexes;
- Lean, mathlib, Pantograph, PeTTa, MORK, and PLN revisions;
- tactic edges and diagnostics;
- certificates and replay runs;
- experiments, critics, literature notes, and human guidance.

This makes the combined system scientifically inspectable. A later model can learn not only that a theorem was proved, but which premise set, tactic policy, creative mechanism, and formal obstruction led to it.

5.7 Speculative hypothesis layer

Hyperon and LLM suggestions first enter a speculative layer. Examples include:

- missing-lemma abductions;
- analogy mappings;
- representation changes;
- stronger but cleaner claims;
- candidate counterexamples;
- proposed induction invariants;
- premise clusters likely to unlock a formal state.

Speculative objects can influence retrieval and scheduling. They cannot be used as established dependencies until a commit-gate workflow promotes them conservatively.

5.8 Ground-prefix atom schema

Put stable namespace fields first so common MORK queries share long ground prefixes:

```

(proof p17 layer committed research-state rs101)
(proof p17 layer committed research-move rm204)
(proof p17 out rs101 proposes rm204 re901)

(proof p17 layer committed formal-declaration fd17)
(proof p17 formal-environment fd17 env9)
(proof p17 formal-root-state frun31 fs1)
(proof p17 formal-state-status fs1 open)
(proof p17 out fs1 tactic-application ta8 fe44)
(proof p17 out ta8 requires fs2 fe45)
(proof p17 out ta8 requires fs3 fe46)

(proof p17 layer committed claim c42)

```

```
(proof p17 layer committed alignment al9)
(proof p17 alignment-claim al9 c42)
(proof p17 alignment-declaration al9 fd17)
(proof p17 alignment-status al9 aligned)
```

Forward and reverse edge atoms may be duplicated as generated indexes. They are projections from one semantic edge ID, not independent assertions.

5.9 Exact hashes and semantic signatures

Formal state identity needs two related fingerprints.

Exact state hash. Includes the concrete Lean target, local context, relevant environment identity, and serialization version. It supports safe replay and exact deduplication.

Semantic state signature. Normalizes selected presentational details and may include structural embeddings. It proposes possible transpositions or alpha-equivalent states.

The semantic signature never merges nodes by itself. Concrete local names, implicit arguments, metavariables, typeclass instances, and environment changes can affect tactic applicability. A merge requires an exact match or an explicit equivalence record accepted by the formal layer.

5.10 Local cache and persistent projection

A formal ATP service needs fast mutation and rank propagation. It may use its existing Python hypergraph as the run-local cache. The persistent projection should contain:

- the root and important formal states;
- every tactic edge needed by a returned certificate;
- all open frontier states at a checkpoint;
- dead-edge diagnostics according to retention policy;
- score snapshots and model provenance;
- exact trace chunks stored by hash;
- transposition proposals and accepted merges;
- run status and budget use.

High-frequency intermediate states can be batched into trace artifacts rather than committed one event at a time. The checkpoint manifest must be sufficient to resume and audit.

5.11 Routes and transpositions

A state ID is not a route. Different research strategies or tactic paths may reach the same state. Attempts therefore record both state identity and route identity. This preserves route-specific assumptions, score history, and provenance while allowing shared lemmas and formal states.

MORK's internal PathMap paths are implementation-level indexing paths, not semantic proof routes. Route projections are explicit derived objects.

5.12 Taint and reopening

When a claim, alignment, environment, or formal dependency is retracted or changed:

1. mark direct dependents tainted;
2. propagate taint through the justification graph;
3. mark affected formal certificates stale if dependency or environment hashes no longer match;
4. reopen research states and formal states whose closure depended on the stale object;

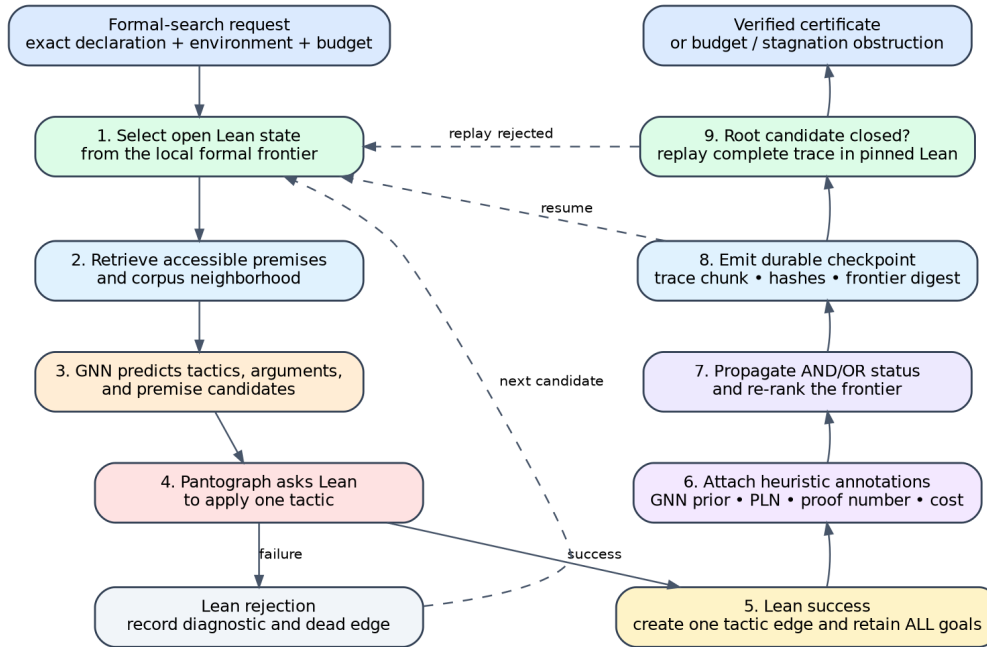
5. preserve historical attempts and old statuses for audit.

MM2 rules may derive the affected cone, but the commit gate records the actual protocol transitions.

6. The Lean-Native Formal ATP Service

6.1 Purpose

The formal ATP service is the principal integration point for the Mathematics-ai-atp code. It packages local proof search behind a stable request/result interface. The service may initially run in the same Python process as the current prover, but it should behave as an external worker from the proof-store's perspective.



Formal ATP inner loop

6.2 Formal-search request

A request contains:

```

{
  "proof_id": "p17",
  "claim_id": "c42",
  "formal_declaration_id": "fd17",
  "run_id": "fr31",
  "base_revision": 982,
  "lease_id": "lease771",
  "fencing_token": 14,
  "lean_source_artifact": "art-source-17",
  "environment_id": "env9",
  "environment_hash": "sha256:...",
  "corpus_manifest": "corpus-lean-dojo-4",
  "model_bundle": "models-atp-2026-07",
  "search_policy": "gnn-pln-best-first-v1",
  "budget": {
    "wall_seconds": 1800,
    "max_nodes": 10000,
    "max_depth": 64,
    "checkpoint_every_nodes": 250
  }
}

```

```
}
}
```

The request is immutable. A resumed run receives a new budget extension record but preserves its original declaration and environment identity.

6.3 Tactic and premise policies

The formal service can use the existing GNN components for:

- tactic-name prediction;
- argument prediction;
- premise or lemma retrieval;
- prior probabilities over actions.

The protocol does not prescribe one neural architecture. It records the model artifact hash, configuration, training-data manifest, inference code version, and output scores. This makes it possible to compare GNN, transformer, retrieval-augmented, or LLM-based policies under the same formal search and replay contract.

LeanDojo-style corpora are especially useful because they expose proof states, tactics, and premise annotations and support programmatic Lean interaction.⁹ A current deployment should pin the exact corpus and extraction version rather than use the name “LeanDojo” as a vague provenance label.

6.4 Pantograph and exact state transitions

Pantograph receives the exact Lean state and tactic command and returns success, diagnostics, and every resulting Lean goal.¹⁰ The ATP service translates that result into one tactic edge.

```
Lean rejects tactic
-> dead edge + diagnostic artifact
```

```
Lean accepts tactic and returns zero goals
-> childless solved edge
```

```
Lean accepts tactic and returns k goals
-> one edge with exactly k required children
```

The list of subgoals is part of the semantic event. It cannot be truncated for convenience. A `top_k_subgoals` configuration may control how many children are expanded concurrently, but not how many are stored as obligations.

6.5 Local AND/OR search

The current Mathematics-ai-atp hypergraph already captures the essential local topology: tactics are OR alternatives and tactic-generated subgoals are AND children.¹¹ Version 3 generalizes the persistent contract to support:

- multi-parent transpositions;
- exact and semantic state signatures;
- resumable checkpoints;
- route-aware statistics;
- proof and disproof numbers;
- several model policies;

⁹ K. Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, arXiv:2306.15626, and LeanDojo project resources: <https://arxiv.org/abs/2306.15626>

¹⁰ L. Aniva et al., *Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4*, and PyPantograph: <https://github.com/stanford-centaur/PyPantograph>

¹¹ iCog-Labs-Dev, `maths_ai/hybrid_reasoner/hypergraph.py`, main branch, reviewed 16 July 2026: https://github.com/iCog-Labs-Dev/Mathematics-ai-atp/blob/main/maths_ai/hybrid_reasoner/hypergraph.py

- complete trace export;
- independent replay.

The first adapter does not need to implement every generalization. It must, however, preserve the protocol invariants so future upgrades do not change proof meaning.

6.6 PLN's local role

PLN may attach strength and confidence to relations among the local goal, hypotheses, candidate premises, tactic outcomes, and analogous states. It may help rank:

- which open formal state to expand;
- which tactic edge to try first;
- which premise cluster is relevant;
- which subgoal is likely to be the bottleneck;
- which formal failure resembles a known obstruction.

PLN truth values remain heuristic. In particular, a fallback or sampled score is exploration metadata, not evidence that a goal is provable. The protocol never maps a numeric threshold directly to `formal-closed`.

6.7 Parent goal is context, not an asserted theorem

A subgoal is generated from a parent goal by a tactic. That relation should be represented explicitly in the formal graph. The parent target is not automatically an established hypothesis of the child. Only the actual Lean local context is an authoritative premise set.

A PLN adapter may use the parent goal as contextual metadata, but it must not silently assert the unproved parent proposition at strength one and then interpret a high derived score as evidence for the child. This rule prevents circular heuristic support.

6.8 Search features and backup

A formal node and edge may carry:

```
gnn_tactic_prior
argument_probability
premise_relevance
pln_strength
pln_confidence
proof_number
disproof_number
depth
estimated_execution_cost
state_novelty
transposition_count
failure_family
```

One baseline edge utility is:

$$[U(e) = w_g G_e + w_p P_e + w_n N_e - w_c C_e - w_f F_e,]$$

where the terms are normalized action prior, PLN or premise support, proof-number progress, cost, and repeated-failure penalty. The precise formula is policy configuration. It is never part of the proof certificate.

For AND/OR backup, a simple proof-number-inspired scheme is:

```
OR proof estimate      = min(child proof estimates)
AND proof estimate     = sum(child proof estimates)
OR disproof estimate   = sum(child disproof estimates)
AND disproof estimate  = min(child disproof estimates)
```

Learned value estimates or PUCT may be added after sufficient search data exists.¹²

6.9 Formal checkpoints

A checkpoint includes:

- run ID and local sequence number;
- root formal state ID;
- base project revision and lease token;
- environment and model hashes;
- formal states added or changed since the prior checkpoint;
- tactic edges and complete child lists;
- frontier digest and selected score snapshots;
- closed, dead, and open state counts;
- repeated failure signatures;
- trace chunk artifacts;
- budget consumed and remaining;
- candidate transpositions;
- whether a complete proof candidate exists.

Checkpoints are appended to the journal as project events or referenced from a compact event. The exact trace data may live in the artifact store.

6.10 Formal result dispositions

A formal run returns one of:

```
proved-pending-replay
budget-exhausted
stagnated
counterexample
invalid-request
environment-error
internal-error
```

Partial progress is represented by formal checkpoints and may include closed local states or verified sublemmas even when the root remains open. `stagnated` must include a typed obstruction capsule rather than a mere message saying “could not prove.” Cancellation and staleness are run lifecycle statuses rather than claims of mathematical disposition.

6.11 Independent replay

The replay service reconstructs the declaration under the pinned environment and checks the exact proof term or tactic trace. It records:

- Lean toolchain and mathlib revisions;
- source and import hashes;
- declaration name and theorem hash;
- dependency declaration hashes;
- tactics or proof term;
- axioms used;
- sorry or placeholder policy;
- stdout, stderr, timing, and exit status;
- replay source artifact and certificate digest.

¹² G. Lample et al., *HyperTree Proof Search for Neural Theorem Proving*, arXiv:2205.11491; <https://arxiv.org/abs/2205.11491>

Only a successful replay can produce a committed formal-certificate-valid relation.

6.12 Required hardening tests

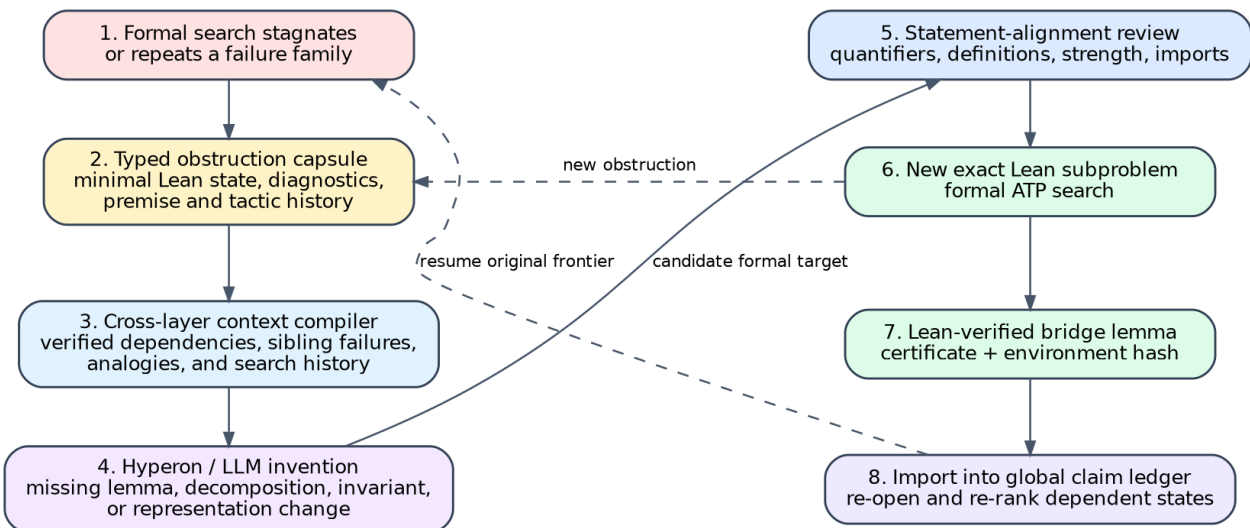
Before the ATP service is treated as authoritative, it should pass at least these tests:

A tactic returning five goals creates five required children.
 A heuristic score of 1.0 cannot close a formal state.
 A fallback or random score cannot close a formal state.
 A childless failed tactic cannot be confused with a successful zero-goal transition.
 A proof with one omitted child is rejected.
 A certificate under the wrong environment hash is stale.
 A replay using sorry is rejected under production policy.
 A semantic state match cannot merge exact states without an accepted equivalence.
 A late formal result cannot commit under an obsolete fencing token.

7. The Bidirectional Lemma and Obstruction Bridge

7.1 Why the bridge is the central integration

The most valuable collaboration is not an LLM choosing the next tactic. It is the creation of new mathematical objects that change the formal search space, together with exact feedback showing which objects are actually needed.



Bidirectional lemma and obstruction bridge

7.2 Research-to-formal messages

The outer loop may send:

- a bridge lemma;
- a decomposition into several exact claims;
- a stronger but structurally cleaner lemma plus a specialization path;
- an induction invariant;
- a representation change;
- a restricted or generalized statement;
- an explicit premise neighborhood;
- a suggested library abstraction;

- a counterexample-search target;
- a proof sketch translated into tactic families.

Every message is bound to a claim ID, alignment record, theorem revision, and explicit relation to the literal target.

7.3 Formal-to-research messages

The formal service may return:

- a verified bridge lemma;
- a verified special case;
- a partially closed AND decomposition;
- the smallest persistent open state;
- a cluster of repeated tactic failures;
- missing accessible premises;
- a typeclass, coercion, universe, or elaboration obstruction;
- evidence that the formal statement is too strong;
- a likely definition mismatch;
- a tactic path that nearly closes the goal;
- premises repeatedly selected by the model;
- a concrete counterexample obtained through evaluation;
- an environment or library incompatibility.

7.4 Typed obstruction schema

An obstruction record should include:

```
{
  "obstruction_id": "obs44",
  "kind": "missing-lemma",
  "formal_run_id": "fr31",
  "formal_state_ids": ["fs88", "fs91"],
  "minimal_state_artifact": "art-state-88",
  "diagnostic_artifacts": ["art-diag-9"],
  "repeated_failure_family": "rewrite-normalization",
  "premises_considered": ["Nat.add_assoc", "..."],
  "tactic_families_considered": ["simp", "rw", "linarith"],
  "evidence": ["three routes converge on the same residual equality"],
  "suggested_escalation": "invent-normalization-lemma",
  "environment_hash": "sha256:..."
}
```

The taxonomy can include missing-lemma, missing-premise, representation-mismatch, statement-too-strong, statement-too-weak, library-gap, elaboration, typeclass, coercion, resource, search-policy, likely-false, and unknown.

7.5 Missing-lemma abduction

Given an obstruction, Hyperon and an LLM can propose:

1. the weakest lemma that would bridge the exact dependency gap;
2. one or more stronger but cleaner variants;
3. the relation between the lemma and the target;
4. prerequisites and likely library analogues;
5. a cheap falsification or finite test;
6. an exact Lean statement draft;

7. a first formal search budget.

The candidate remains speculative until alignment review and formal verification.

7.6 Statement-alignment review

Before a candidate lemma becomes a formal-search request, a reviewer or specialized worker checks:

- quantifier order;
- domains and coercions;
- hidden finiteness or decidability assumptions;
- universe and typeclass parameters;
- classical versus constructive content;
- whether the Lean statement is stronger or weaker;
- whether the selected definitions match the intended mathematics;
- whether the implication back to the target is explicit.

The review may conclude that the informal claim was underspecified. That is useful progress, not a protocol failure.

7.7 Verified-lemma import

When the formal service verifies a bridge lemma:

1. commit the exact declaration, certificate, and replay;
2. promote the aligned claim according to alignment policy;
3. add the lemma to the verified premise index;
4. update every research and formal state that depends on it;
5. reopen or re-rank previously blocked formal states;
6. record which obstruction it resolved;
7. update mechanism and scheduler statistics.

This turns one formal success into a project-wide resource.

7.8 Representation-change loop

Some failures cannot be repaired by one lemma. The formal obstruction may show that the current encoding generates unmanageable coercions, hides an inductive structure, or lacks a library abstraction. The bridge can then propose a new formal representation with an explicit equivalence task.

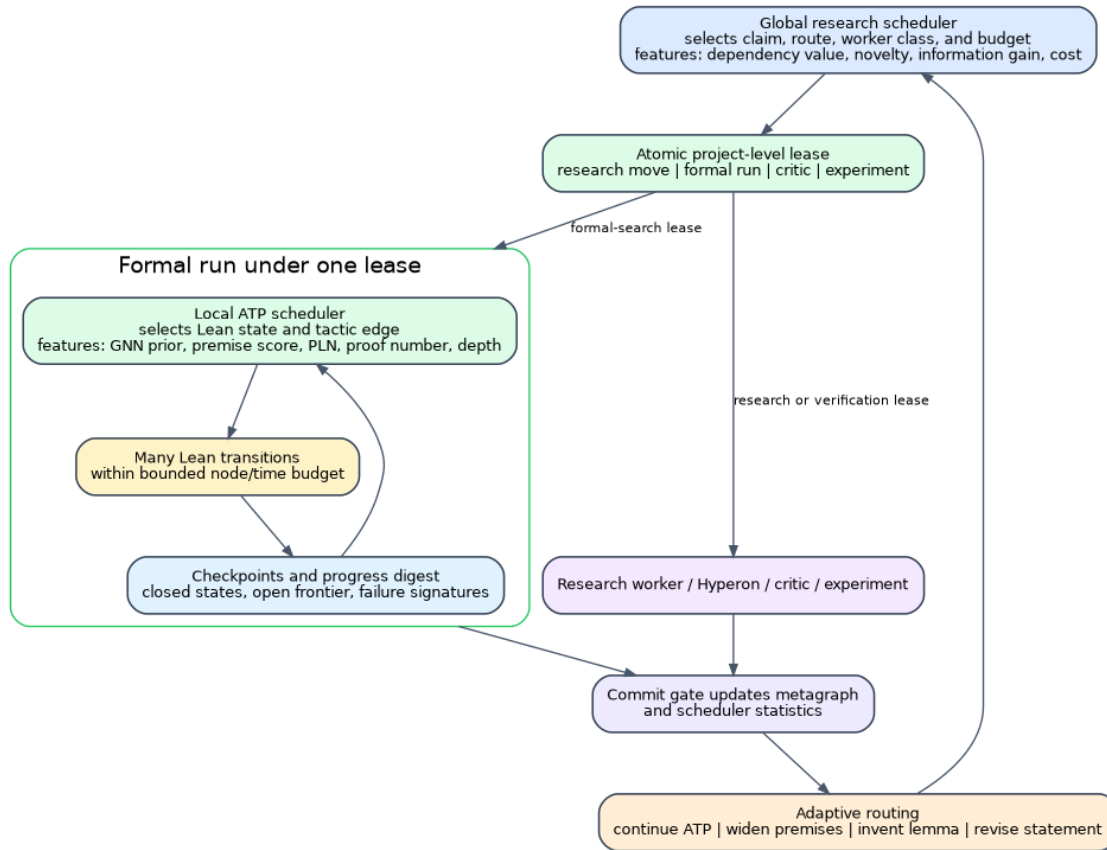
The old and new declarations remain distinct. The graph records:

```
reformulates
isomorphic-to
implies
specializes-to
preserves-target-under
```

A representation change is not allowed to erase the original theorem identity.

8. Hierarchical Scheduling, Leasing, and Parallel Search

8.1 Two schedulers



Hierarchical scheduling

The **global scheduler** chooses among research moves, formal claims, critics, experiments, and audits. The **local ATP scheduler** chooses among exact Lean states, tactics, arguments, and premises inside one formal run.

The global scheduler leases a formal run, not every tactic application. The formal scheduler returns checkpoints and objective progress signals.

8.2 Global scheduling features

A global move may be scored by:

- dependency centrality
- expected theorem impact
- expected information gain
- novelty and mechanism diversity
- verification value
- formalization readiness
- estimated cost
- repeated-failure risk
- human priority
- availability of suitable worker/model/tool

A low-probability counterexample search may have high information value. A formalization attempt may be worthwhile even if immediate proof is unlikely because it exposes hidden assumptions.

8.3 Local scheduling features

A local formal action may use:

```
GNN tactic prior
argument probability
premise retrieval score
PLN relevance
proof/disproof number
state depth and size
transposition reuse
execution cost
failure-family penalty
policy diversity quota
```

The local scheduler should remain reproducible enough that an evaluation can explain why one state was expanded before another.

8.4 Atomic leasing

The only safe global selection primitive is atomic:

```
proof-lease-next(proof_id, worker_class, ttl)
```

Under the proof lock, the service expires stale leases, constructs the eligible frontier, selects a move and route, increments a fencing token, records the score snapshot, appends the event, updates MORK, and returns the lease.

A formal result includes the lease ID, fencing token, and base revision. A late result is preserved but cannot commit under an obsolete token.

8.5 Formal run leases and subleases

A formal run may internally parallelize several Lean state expansions. Those are run-local subleases and do not need to become project-level leases unless they are distributed across processes. The formal adapter is responsible for avoiding duplicate local work and for emitting a consistent checkpoint.

8.6 Cognitive and policy diversity

Parallelism is useful when workers differ. A three-worker project-level portfolio might reserve capacity for:

- one exploitation task, often the highest-value formal or research move;
- one exploration or analogy task;
- one critic, counterexample, experiment, or replay task.

Within the formal ATP service, diversity can mean different tactic models, premise retrievers, temperature settings, tactic-family quotas, or novelty-biased searches. The graph records which policy produced each edge.

8.7 Adaptive routing

A baseline router is:

```
Formal ATP is closing states at a healthy rate
-> continue formal search.
```

```
Repeated failures suggest the premise neighborhood is too narrow
-> widen retrieval or use a different premise policy.
```

```
Several tactic families reach the same residual obstruction
-> emit a missing-lemma or representation-change request.
```

The formal statement appears stronger than the reviewed claim

-> return to statement alignment.

A complete trace exists

-> run independent replay.

A verified lemma resolves several blocked states

-> import globally and re-rank all dependent frontiers.

8.8 Progressive widening and learned scheduling

Initially, both schedulers should use transparent best-first policies. After collecting enough attempt data, the system may add:

- PUCT exploration terms;
- progressive widening for dynamically generated research moves and tactics;
- learned outcome values;
- proof-number search;
- transposition statistics;
- portfolio bandits over model and persona choices.

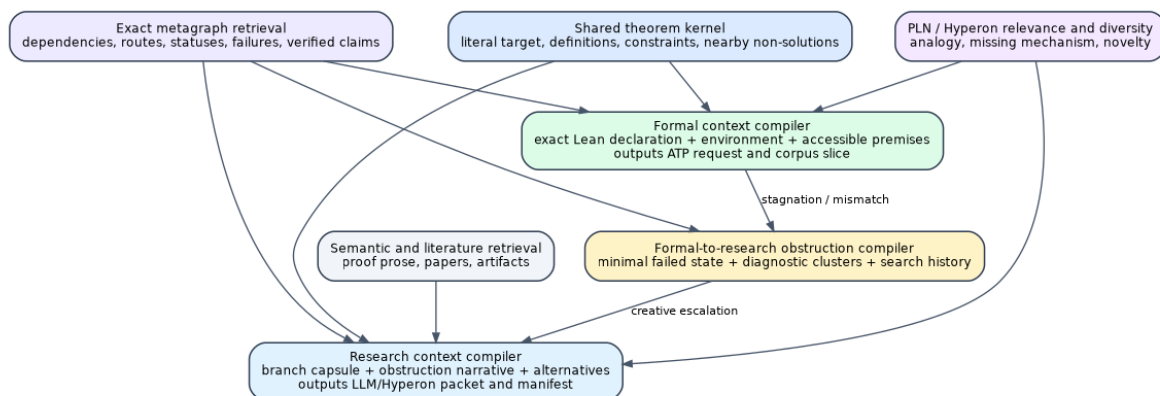
Objective outcomes should dominate self-reported confidence:

- formal states closed;
- verified claims produced;
- branches pruned;
- counterexamples found;
- critics survived;
- obstructions clarified;
- reusable lemmas imported;
- replay success;
- cost and latency.

9. Context Compilation and Proof Memory

9.1 Context compilation is not transcript summarization

Once proof state is externalized, each worker needs a bounded view of the project. The correct view differs by role. The system therefore has multiple context compilers over one source graph.



Dual context compilation

9.2 Shared theorem kernel

Every mathematical worker receives a canonical theorem kernel containing:

- literal theorem statement;
- definitions and notation;
- quantifier order and domains;
- allowed background results;
- constructivity, effectivity, and complexity requirements;
- exact theorem hash and revision;
- nearby statements and bypasses that do not suffice;
- aligned formal declarations, when any.

9.3 Research context packet

A research packet contains:

- active research state and route;
- selected move;
- established, provisional, and speculative claims with visible statuses;
- open goals and their AND/OR semantics;
- latest critic, experiment, or formal obstruction;
- sibling failures and taboo mechanisms;
- reusable verified lemmas;
- relevant literature and artifacts;
- Hyperon analogies or missing-lemma hypotheses;
- one deliberately dissimilar strategy when the branch is stale;
- an output schema and tool policy.

9.4 Formal context packet

A formal request contains machine-oriented context rather than a long narrative:

- exact Lean source and declaration;
- pinned environment manifest;
- accessible imports and premise universe;
- selected verified lemmas and corpus slice;
- model bundle and search policy;
- exact root state artifact;
- formal budget and checkpoint policy;
- prior formal runs and reusable states;
- accepted transpositions;
- forbidden axioms or placeholders.

The ATP adapter may compile its own tensors and indexes from this packet, but the provenance points back to the manifest.

9.5 Formal-to-research obstruction packet

When formal search escalates, the compiler does not send the full hypergraph to an LLM. It includes:

- the minimal exact failed state or small state cluster;

- a human-readable rendering of local hypotheses and target;
- tactic and premise families tried;
- repeated diagnostic signatures;
- closest nearly closed paths;
- verified dependencies available;
- suspected missing lemma or mismatch type;
- links to full artifacts;
- explicit questions for the creative worker.

9.6 Candidate retrieval channels

Research context candidates come from:

1. exact MORK structural retrieval;
2. PLN and Hyperon uncertain relevance;
3. semantic and full-text retrieval over prose and literature;
4. deliberate diversity;
5. formal-search statistics and obstruction similarity.

Formal premise candidates come from:

1. Lean accessibility and import rules;
2. verified claim dependencies;
3. corpus retrieval and GNN premise ranking;
4. local tactic history;
5. analogy to successful formal states;
6. human or research-worker premise suggestions.

9.7 Diversity-aware selection

A transparent baseline research score is:

$$[S_i = 4D_i + 3A_i + 2.5E_i + 2V_i + 1.5F_i + 1.3P_i + N_i - 1.5R_i - C_i,]$$

where dependency relevance, ancestry, semantic similarity, verification, matching failure mode, PLN relevance, novelty, redundancy, and token cost are recorded. Maximal marginal relevance or another diversity-aware selection prevents a context packet from becoming five paraphrases of one idea.

9.8 Context manifest

Every packet has a manifest with:

- proof, state, move, claim, formal run, and lease IDs;
- theorem and environment hashes;
- compiler version and token budget;
- every source ID and version;
- retrieval channel, score, status, and inclusion reason;
- high-scoring exclusions and reasons;
- final packet digest;
- model-facing rendering version.

The manifest permits exact reproduction and later analysis of context misses.

9.9 Context feedback

Workers may cite which sources were useful. Critics may identify missing context. Formal runs can report that a proposed premise was inaccessible or irrelevant. These records update retrieval weights and Hyperon guidance rules without changing proof history.

10. Autonomous Proof Cycles and Escalation Policy

10.1 The finite unit of autonomy

OmegaClaw normally invokes:

```
proof-cycle(proof_id)
```

A cycle is finite and auditable. Persistent autonomy emerges through repeated cycles under budget and policy limits.

10.2 Reference outer-cycle algorithm

```
def proof_cycle(proof_id: str) -> CycleDigest:
    lease = store.lease_best_global_move(proof_id)
    if lease is None:
        return audit_or_expand_frontier(proof_id)

    if lease.worker_class == "formal-atp":
        request = formal_context.compile_request(lease)
        result = formal_adapter.run_or_resume(request)
    else:
        packet = research_context.compile(lease)
        result = dispatcher.run(lease.worker_class, packet)

    store.save_exact_result_and_artifacts(lease, result)
    proposal = validator.parse(result)
    reviews = verification_policy.run_required_checks(proposal)
    commit = commit_gate.apply(
        proposal=proposal,
        reviews=reviews,
        expected_revision=lease.base_revision,
        lease_id=lease.lease_id,
        fencing_token=lease.fencing_token,
    )
    maintenance.update_views(commit)
    scheduler.update_statistics(commit)
    return commit.digest
```

10.3 Formal run cadence

A formal run may return a checkpoint rather than terminate. The global scheduler can:

- extend its budget;
- pause it while a bridge lemma is investigated;
- fork it under another model policy;
- cancel it;
- import its partial verified lemmas;
- mark it stale after a theorem or environment change.

The coordinator receives compact digests, not every local transition.

10.4 Stagnation policy

A baseline policy is:

After one failed research attempt:
 classify the earliest obstruction and retry only with a concrete repair.

After two non-progress attempts:
 generate several genuinely different mechanisms and queue viable ones.

After repeated formal failures in one tactic/premise family:
 diversify the formal policy or widen retrieval.

After several policy families converge on the same residual state:
 create a bridge-lemma or representation-change task.

After formal statement mismatch evidence:
 require statement-alignment review before more search.

After root-level global stagnation:
 run an audit, literature refresh, Hyperon intervention, and theorem review.

10.5 The five-mechanism operator

A structured creativity request asks for five different mechanisms. Each must state:

- cognitive operator and technique family;
- central idea;
- exact bridge lemma or reduction;
- why it would close or clarify the current gap;
- prerequisite claim IDs;
- expected formal manifestation;
- earliest inexpensive falsification test;
- information gain if false;
- promise, cost, novelty, and risk;
- relation to existing branches;
- whether it proves, bypasses, generalizes, or reframes the target;
- first executable research or formal step.

Technique tags and graph distance enforce diversity.

10.6 Critic and replay policy

A critic is required before a reusable informal claim becomes a strong premise. Lean replay is required before a formal certificate changes formal status. A completion candidate for the literal theorem receives:

- theorem-identity audit;
- dependency closure audit;
- statement-alignment audit;
- artifact and environment audit;
- independent Lean replay of every formal component;
- explicit accounting of any nonformal dependencies.

10.7 Milestone communication

The human receives messages for durable events such as:

- a verified or critic-accepted central lemma;
- a major route opened, refuted, bypassed, or superseded;
- a formal obstruction that changes strategy;

- a statement mismatch requiring semantic judgment;
- a reproducible counterexample;
- a proof-complete candidate;
- a resource or security decision.

Routine tactic expansions and checkpoint details remain in the workspace.

11. Epistemic Status, Formal Soundness, and Verification

11.1 Two related status systems

Research claims use statuses such as:

conjectural
empirical
provisional
critic-accepted
formally-closed
lean-verified
refuted
retracted
stale

Formal states use operational statuses such as:

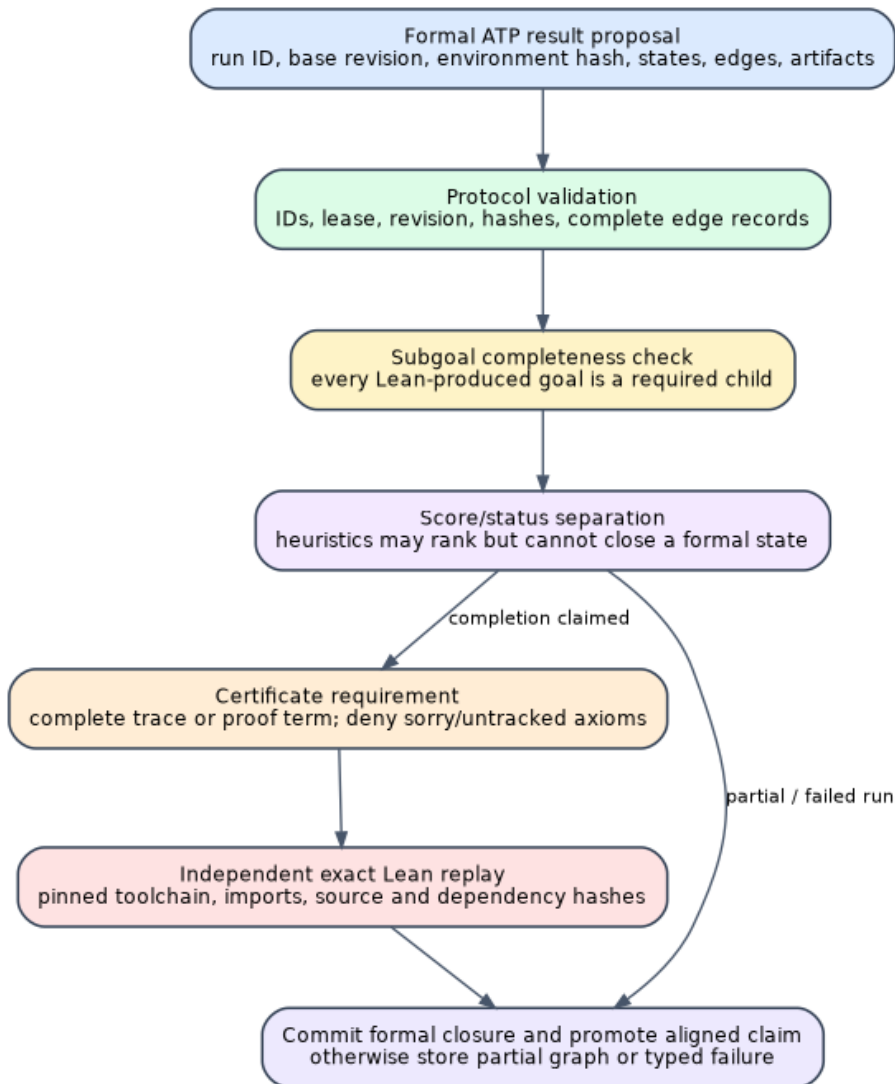
open
expanded
formally-closed
lean-verified
failed
pruned
stale

Formal declarations and certificates use:

draft
aligned
searching
certificate-produced
replay-pending
replay-accepted
replay-rejected
stale

The systems are linked but not identical.

11.2 Formal soundness gate



Formal result validation and replay gate

A formal result proposal passes through:

1. schema, namespace, revision, lease, and artifact validation;
2. complete tactic-edge validation, including every Lean-produced child;
3. score/status separation checks;
4. certificate and environment checks;
5. independent Lean replay;
6. statement-alignment policy;
7. commit or conservative storage as partial/failure output.

11.3 Leaf closure

Only two leaf cases exist:

- Lean accepts a tactic and returns zero goals: solved leaf;
- Lean rejects the tactic or an execution error occurs: dead edge.

A missing backend, timeout, empty model output, or parser failure is not a zero-goal success.

11.4 Dependency discipline

A certificate records every formal dependency and environment version. A claim cannot remain `lean-verified` if a dependency is retracted, changed, or imported under a different declaration hash. It becomes stale until replay succeeds under the current dependency set.

11.5 Numerical and symbolic evidence

Evidence	Permitted conclusion
Random sampling	empirical support or a counterexample
Exhaustive finite search	theorem for the enumerated finite domain
Exact symbolic computation	equality or certificate under recorded semantics
Interval arithmetic	bounded numerical statement under recorded precision
Checkable certificate	claim conditional on the independent checker
Heuristic optimizer	candidate or bound, not a proof without certification
PLN or model score	scheduling evidence only

11.6 Bypasses and nearby theorems

A route-level substitute may achieve the practical objective while the literal target remains open. The graph represents bypasses or strengthens-route explicitly. A verified nearby theorem closes the literal target only if an accepted implication chain is present.

11.7 Formalization mismatch

If the first Lean statement differs materially from the informal claim, the result is not treated as a failed proof attempt alone. It is a statement-alignment event. The system records the difference, updates the research state, and may revise definitions or split the claim.

11.8 Auditability

For every promoted formal claim, a human should be able to navigate:

```
claim
-> alignment record
-> Lean declaration and source hash
-> formal run
-> tactic trace or proof term
-> environment manifest
-> replay record
-> dependencies
-> model and corpus provenance
```

12. OmegaClaw, ThreadKeeper, Hyperon, and Skill Architecture

12.1 OmegaClaw coordinator

OmegaClaw is the outer coordinator and user-facing research manager. Its prompt emphasizes policy rather than carrying the full proof. The coordinator reads compact status and milestone digests, invokes `proof-cycle`, requests audits, and surfaces semantic decisions.

OmegaClaw’s published architecture is skill-oriented and operates over PeTTa, which makes narrow proof-service wrappers a natural integration point.¹³

12.2 Minimal skill surface

The main catalogue should expose:

```
(proof-create $spec $workspace)
(proof-cycle $proof $workspace)
(proof-status $proof $workspace)
(proof-audit $proof $workspace)
(proof-export $proof $destination $workspace)
(formal-search $proof $claim $budget $workspace)
(formal-resume $proof $run $budget $workspace)
(statement-align $proof $claim $lean_source $workspace)
(lemma-bridge $proof $obstruction $workspace)
```

Lower-level lease and commit operations remain available for debugging and specialized orchestration.

12.3 ThreadKeeper workers

ThreadKeeper supplies bounded, role-specific dispatch. The mapping is:

```
one proof project      -> one persistent workspace and MORK namespace
one research move     -> one bounded ThreadKeeper dispatch
one formal run        -> one ATP service job, not a long-lived chat
one returned digest   -> status line and full-result pointer
```

The full mathematical result lives in attempt files. Workers do not share unfiltered conversations and do not mutate MORK directly.¹⁴

12.4 Worker roles

Role	Objective	Typical outputs
Explorer	advance one research move	claims, proof sketch, new moves
Lemma inventor	respond to formal obstruction	candidate lemma and Lean draft
Statement-alignment reviewer	compare informal and formal statements	alignment verdict and differences
Critic	find earliest exact gap	verdict, counterexample, repair
Analogy explorer	transfer a mechanism	mapped concepts and bridge task
Experimenter	run preregistered computation	results, certificate, limits
Formal obstruction analyst	explain formal stagnation	minimal failure and escalation
Adjudicator	compare conflicting reports	evidence-weighted decision

The GNN/PLN formal ATP service is not modeled as an LLM persona. It has a machine contract and its own run lifecycle.

12.5 Hyperon as creative and cross-layer partner

Hyperon has three roles:

1. exact and uncertain relevance for context and scheduling;
2. diagnosis of repeated failures and missing dependencies;
3. generation of new mathematical hypotheses and representations.

¹³ ASI Alliance, OmegaClaw-Core project documentation: <https://github.com/asi-alliance/OmegaClaw-Core>

¹⁴ H. L. Greenblatt et al., ThreadKeeper architecture and subagent design; the version 2 package used the public ThreadKeeper branch as an integration reference.

Because the proof state already lives in MORK, Hyperon can reason over research states, formal obstruction types, verified lemmas, tactic families, analogy mappings, and human guidance episodes without a lossy periodic export.¹⁵

12.6 Creative operators

The package supports:

- missing-lemma abduction;
- analogy transfer;
- concept blending;
- representation change and dualization;
- obstruction inversion;
- counterexample and extremal construction;
- strengthen-to-simplify;
- generalize then specialize;
- local-to-global transfer;
- random/deterministic hybridization;
- premise-cluster suggestion from formal traces.

Each hypothesis includes source IDs, prerequisites, expected consequences, novelty, support, cost, risk, falsification test, and an expiry or review condition.

12.7 Prompt versioning

Every prompt, skill, ontology, maintenance rule, and formal adapter has a version and content hash. Attempts and runs record them. Regression fixtures include:

- a hidden quantifier mismatch;
- a bypass that must not close the literal target;
- a tactic producing more children than the expansion width;
- a high PLN score on an unproved goal;
- a fake zero-goal result from a missing backend;
- a stale environment replay;
- a useful bridge lemma generated from formal stagnation;
- several queued alternatives where only one is pursued.

13. Reproducible Computation, Artifacts, Models, and Environments

13.1 Content-addressed artifacts

Large objects remain outside MORK and are referenced by digest:

- theorem prose and definitions;
- contexts, prompts, and responses;
- Lean source, state snapshots, traces, and diagnostics;
- model weights and configuration;
- corpus manifests and retrieval indexes;
- Python, CAS, SAT/SMT, and experiment files;
- papers and literature notes;

¹⁵ TrueAGI, MeTTa-native PLN for Hyperon: <https://github.com/trueagi-io/PLN>

- replay outputs and certificates.

13.2 Formal environment manifest

A formal environment record includes:

```
{
  "environment_id": "env9",
  "lean_version": "...",
  "mathlib_revision": "...",
  "pantograph_revision": "...",
  "lake_manifest_digest": "sha256:...",
  "imports": ["Mathlib.Algebra..."],
  "source_tree_digest": "sha256:...",
  "deny_sorry": true,
  "allowed_axioms": [],
  "platform": {"os": "...", "arch": "..."}
}
```

A model bundle similarly records weights, code, training-data manifest, feature vocabulary, and inference configuration.

13.3 Experiment specification

Every experiment begins with a specification of:

- mathematical question;
- competing hypotheses;
- domain and parameter space;
- exhaustive or randomized method;
- seed policy;
- expected outcomes;
- success criteria;
- inferential limits;
- resource budget;
- requested artifacts.

The result is interpreted conservatively according to the evidence table in Section 11.

13.4 Sandboxing

Research tool workers and formal runs should use:

- no network by default;
- read-only access to declared inputs;
- write access only to the attempt or run directory;
- CPU, memory, time, and output limits;
- captured stdout and stderr;
- pinned executable and package manifests;
- explicit permission for external literature or package installation.

13.5 Event sourcing and checkpoints

Every committed project change is a revisioned event bundle. Formal checkpoints may reference large trace chunks by hash. A clean MORK space can be reconstructed by replay, and derived indexes can be recomputed.

The commit sequence is:

validate proposal and artifacts
 -> acquire proof lock and recheck revision/lease
 -> append and flush event
 -> apply atom operations to MORK
 -> run bounded maintenance
 -> write checkpoint digest
 -> release lock

If the process dies after journal append, replay completes the mutation.

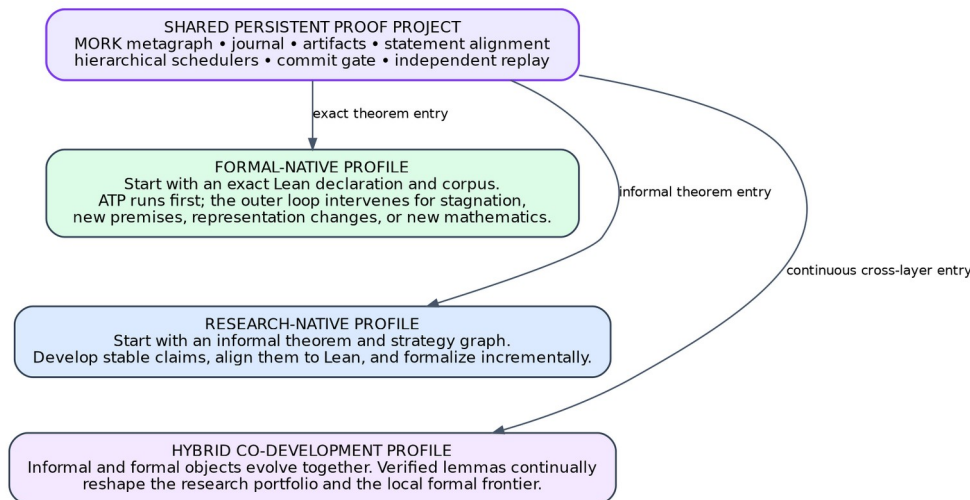
13.6 Deterministic export

A proof export contains:

- normalized theorem specification;
- sorted graph and claim ledger;
- formal declarations and alignment records;
- current research and formal frontiers;
- event and atom schema versions;
- prompt, skill, model, corpus, and environment manifests;
- certificates and replay records;
- artifact manifests and hashes;
- human-readable status report.

14. Operating Profiles and Worked Flows

14.1 Three profiles



Operating profiles

Formal-native profile

Use for LeanDojo benchmarks, formal library theorems, and tasks that already have an exact Lean declaration.

1. Import declaration, environment, and accessible corpus.
2. Create a root formal-search move.
3. Run GNN/PLN/Pantograph search.
4. Replay any candidate certificate.

5. On stagnation, emit a formal obstruction.
6. Invoke the outer loop only when new mathematics, premises, or a new representation is needed.

Easy theorems may complete without any LLM worker.

Research-native profile

Use for open-ended mathematical programs.

1. Create an informal theorem kernel.
2. Build a research strategy graph.
3. Use LLMs, Hyperon, critics, and experiments to develop stable claims.
4. Create alignment records for selected claims.
5. Invoke formal ATP on those subproblems.
6. Import verified lemmas and precise failures.
7. Continue until the literal theorem or a decisive obstruction is established.

Hybrid co-development profile

Use when a theorem is partly formal and partly exploratory. Formal and informal objects evolve together. The main theorem may remain informal while central lemmas are checked, or an exact Lean target may motivate new informal mathematics.

14.2 Formal-native worked flow

Suppose the project imports a LeanDojo theorem.

```
Root claim C0
-> aligned declaration D0
-> formal run FR1
-> GNN predicts constructor tactic
-> Lean returns two subgoals S1 and S2
-> both are retained as required children
-> S1 closes through a library lemma
-> S2 stagnates after several premise policies
-> obstruction OBS2: missing monotonicity bridge
-> LLM/Hyperon proposes lemma C1
-> alignment review creates D1
-> formal run FR2 verifies D1
-> C1 is imported as a verified premise
-> FR1 resumes and closes S2
-> exact replay verifies D0
-> C0 promoted according to alignment policy
```

14.3 Research-native worked flow

Suppose the project begins with a speculative combinatorial theorem.

Outer loop identifies a probabilistic route and a deterministic route.

An experiment falsifies a naive strengthening.

Hyperon proposes a decomposition into an algebraic shield and a random core.

A critic accepts the decomposition conditionally on two lemmas.

The first lemma is formalized and solved by ATP.

The second formalization reveals a hidden finiteness assumption.

The theorem kernel is not silently changed; a statement review is opened.

A corrected lemma is verified and imported.

The research route advances while the original false strengthening remains refuted.

14.4 Bridge-lemma worked flow

A formal run repeatedly reaches goals of the form:

```
h : Normalized x
|- normalize (combine x y) = combine x (normalize y)
```

The obstruction compiler groups several such states and notes that rewriting, simplification, and congruence tactics all stall. Hyperon maps the pattern to a normalization homomorphism. The LLM proposes an exact homomorphism lemma and a weaker local variant. Statement review selects the weaker variant. ATP proves it, replay verifies it, and the lemma is added to the premise index. Several previously blocked states close.

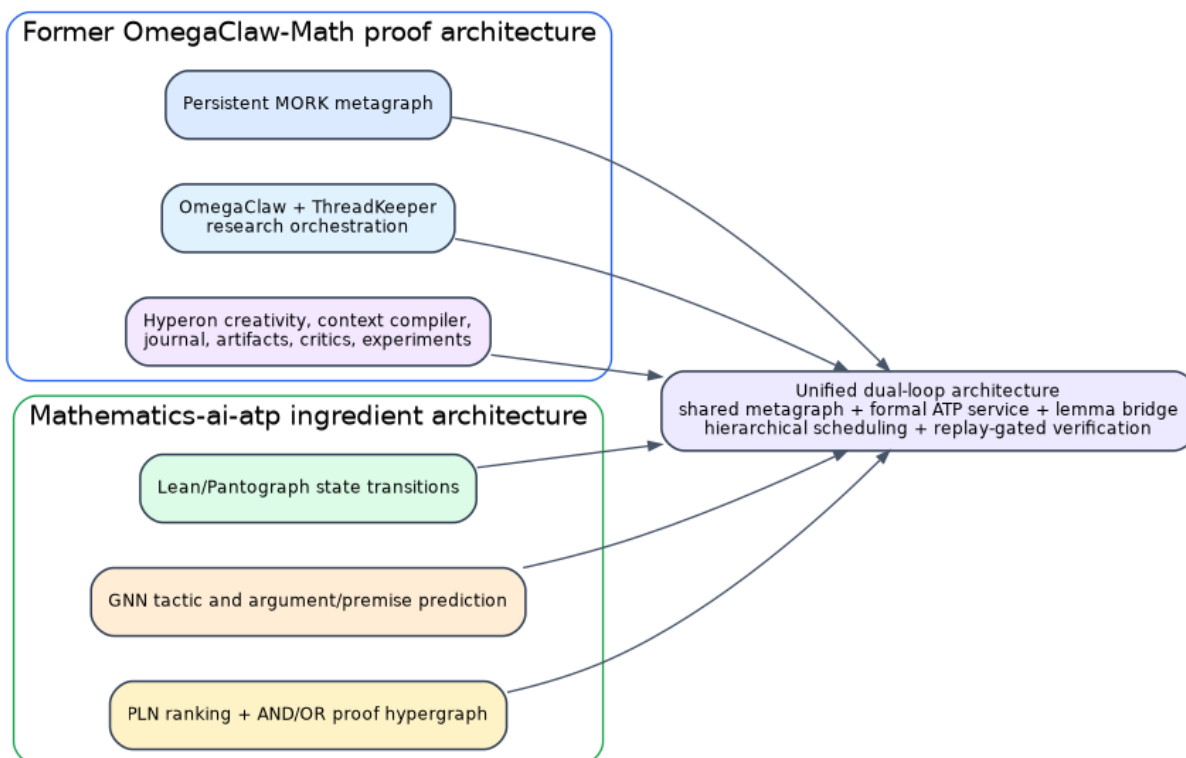
The value came from the interaction: local formal search exposed a common shape; the research layer named the missing abstraction; formal search then certified it.

14.5 N105-N109 legacy fixture

The earlier OmegaClaw-Math package used the N105-N109 proof progression as a realistic long-horizon fixture. It remains useful for the outer loop: conceptual compression, route-level bypass, obstruction inversion, random/deterministic hybridization, artifacts, and a residual open target. Version 3 adds formal subprojects for any stable claims in that route and records exact alignment between the research claims and Lean declarations. The case remains explicitly a fixture for proof dynamics, not a claim that the underlying mathematics has been independently certified by this package.

15. Migration, Deployment, and Compatibility

15.1 Ingredient-to-unified mapping



Migration from the ingredient architectures

The earlier OmegaClaw-Math components map directly:

Earlier component	Version 3 role
MORK research metagraph	shared unified metagraph with added formal and alignment layers
proof-store and event journal	retained and extended with formal checkpoints
OmegaClaw coordinator	global outer-loop coordinator
ThreadKeeper workers	bounded research, critic, and alignment workers
Hyperon creativity	cross-layer hypothesis and obstruction reasoning
incremental Lean formalizer	statement-alignment and formal-request producer

The Mathematics-ai-atp components map as follows:

Current ATP component	Version 3 role
GNN tactic predictor	formal action policy
argument/premise predictor	tactic argument and premise policy
PLN inference	local heuristic annotation and ranking
proof hypergraph	run-local AND/OR cache and persistent formal projection
Pantograph executor	exact Lean transition adapter
proof trace	candidate certificate input to replay

15.2 Integration sequence

A low-risk sequence is:

1. **Wrap without rewriting.** Add a service wrapper around the existing ATP entry point and export formal-search request/result JSON.
2. **Enforce soundness invariants.** Retain all Lean subgoals, prohibit heuristic closure, distinguish failed from zero-goal transitions, and add replay.
3. **Persist checkpoints.** Export formal states, tactic edges, trace chunks, environment and model manifests.
4. **Add the bridge.** Emit typed obstruction capsules and accept aligned bridge-lemma requests.
5. **Share the metagraph.** Project important formal states and results into MORK while retaining the local cache.
6. **Add transpositions and parallel policies.** Introduce exact state sharing, semantic candidates, and policy diversity after the protocol is stable.

15.3 Repository layout

```

omegaclaw-math-unified-atp/
  README.md
  ARCHITECTURE.md
  CHANGELOG.md
  docs/
    TECHNICAL_DESIGN.md
    UNIFIED_METAGRAPH.md
    FORMAL_ATP_INTEGRATION.md
    HIERARCHICAL_SCHEDULING.md
    FORMAL_SOUNDNESS_INVARIANTS.md
    MIGRATION_FROM_V2.md
  figures/
  mork/
    unified_proof_atoms.metta
    formal_indexes.mm2
    dependency_reachability.mm2
    taint_propagation.mm2
    transposition_candidates.mm2
  schemas/
    proof-spec.schema.json

```

```

worker-result.schema.json
formal-search-request.schema.json
formal-search-result.schema.json
formal-state.schema.json
tactic-application.schema.json
statement-alignment.schema.json
obstruction.schema.json
score-vector.schema.json
skills/
  proof-orchestrator/
  formal-atp-search/
  statement-alignment/
  lemma-bridge/
  proof-hyperon-creative/
  proof-critic/
  proof-audit/
python/
  src/mathproof/
    store.py
    formal_atp.py
    formal_protocol.py
    formal_commit.py
    context_compiler.py
    scheduler.py
    ...
  tests/
examples/
tools/

```

15.4 Deployment topology

For a first deployment:

- one proof-store process per active theorem;
- one MORK space owned by that process;
- one local formal ATP service per machine or GPU worker;
- Unix-domain socket or local HTTP transport;
- content-addressed local objects with optional remote backup;
- two or three project-level workers;
- one independent Lean replay process;
- pinned repositories and model artifacts.

At scale, shard by theorem and use a separate global verified-lemma and mechanism service. Keep the same contracts.

15.5 Compatibility with alternative provers

The formal adapter is intentionally generic enough to support another interactive theorem prover if it supplies:

- exact state identity;
- action application;
- complete subgoal lists;
- diagnostics;
- replayable certificates;
- environment manifests.

Lean remains the first implementation and the authority for Lean claims. The unified project can host several formal systems, but cross-system claim alignment is explicit.

15.6 Schema evolution and migration

Version every event, atom, schema, prompt, skill, maintenance rule, model manifest, and environment manifest. A migration emits explicit events or a revision-zero import stream. It never rewrites old event history in place.

16. Evaluation Methodology and Implementation Roadmap

16.1 Evaluation questions

The combined system should be evaluated on more than theorem completion. Core questions are:

- Does formal replay eliminate false-positive completion?
- Does persistence improve resume and long-horizon strategy?
- Do LLM/Hyperon bridge lemmas solve formal states that local ATP cannot?
- Does formal feedback improve research decisions?
- Does hierarchical scheduling reduce cost relative to calling a large LLM on every step?
- Are statement mismatches detected early?
- Are verified lemmas reused across branches?
- Does parallelism increase verified progress rather than duplicated prose?

16.2 Correctness metrics

Metric	Definition
Formal false-positive rate	claimed formal completions rejected by replay
Omitted-subgoal rate	tactic results committed without every Lean-produced child
Heuristic-closure violations	formal states closed by score rather than Lean
Statement-alignment mismatch rate	informal claims whose Lean form differs materially
Environment-staleness rate	certificates invalid under current pinned dependencies
Unsupported-dependency rate	claims using premises not available at stated status
Taint propagation latency	time to reopen all affected states after invalidation
Literal-target false-positive rate	bypasses or nearby theorems misreported as target proof

The first three should be zero by construction.

16.3 Search productivity metrics

- Lean theorems proved per node and per wall-clock hour;
- independent replay success rate;
- formal states closed per run;
- verified bridge lemmas per million model tokens;
- fraction of formal stagnations producing a useful obstruction;
- fraction of bridge hypotheses that survive criticism and formalization;
- reuse count of verified lemmas;
- duplicate state expansion rate;
- transposition reuse rate;
- branches pruned per experiment;
- human interventions per durable milestone;
- cost per replay-verified theorem.

16.4 Context and bridge quality

Measure:

- retrieval precision and dependency omission;
- source-channel contribution;
- context redundancy and token efficiency;
- frequency of inaccessible formal premises;
- obstruction compression ratio from full trace to LLM packet;
- bridge lemma acceptance and proof rate;
- time from obstruction discovery to verified repair;
- rate of speculative suggestions presented incorrectly as facts, which should be zero.

16.5 Ablation ladder

A recommended evaluation compares:

Condition	Capability added
A	Hardened Mathematics-ai-atp formal search alone
B	Persistent formal checkpoints and replay
C	Unified MORK project and global scheduler
D	Research context compiler and critics
E	LLM-generated bridge lemmas and decompositions
F	Hyperon analogy and missing-lemma operators
G	Parallel policy-diverse workers
H	Learned hierarchical scheduling

Use LeanDojo or another pinned formal benchmark for exact proof search and long-horizon fixtures for research management. Compare under matched compute and theorem sets.

16.6 Phase roadmap

Phase 0: protocol and fixtures

- add formal schemas and example traces;
- import a small Lean theorem fixture;
- create tests for omitted subgoals, heuristic closure, replay, and environment drift;
- retain the N105-N109 research fixture.

Phase 1: sound formal adapter

- wrap the current ATP search;
- retain all Lean-generated obligations;
- distinguish tactic rejection from zero-goal success;
- export complete candidate traces;
- add independent replay;
- record model, corpus, and environment manifests.

Exit criterion: no formal status change is possible without a valid Lean transition and replay.

Phase 2: persistent formal graph

- add formal run, state, tactic, and checkpoint atoms;
- store trace chunks by hash;
- resume from checkpoints;
- add exact state hashes and transposition candidates.

Exit criterion: a crash and restart preserves proof meaning and loses at most the configured uncheckpointed search interval.

Phase 3: unified scheduling

- add formal-run move type to the global frontier;
- implement formal progress digests and adaptive budget extension;
- add atomic leases and stale-result handling;
- compare global and local policies.

Exit criterion: two or three workers share a proof without duplicate commits or status races.

Phase 4: lemma and obstruction bridge

- define obstruction taxonomy;
- compile minimal formal-to-research packets;
- add statement-alignment workflow;
- add lemma-inventor and representation-change skills;
- import verified lemmas into formal premise indexes.

Exit criterion: at least one formal stagnation is resolved by a bridge lemma generated through the outer loop.

Phase 5: Hyperon cross-layer creativity

- reason over formal failure signatures and research mechanisms;
- add missing-lemma, analogy, and premise-cluster rules;
- track hypothesis outcomes;
- learn from human guidance.

Exit criterion: Hyperon suggestions are provenance-preserving, nonredundant, and improve some objective search outcomes.

Phase 6: transpositions and policy portfolios

- implement exact multi-parent formal states;
- add semantic transposition proposals;
- run GNN, retrieval, and LLM policy portfolios;
- introduce proof numbers and progressive widening.

Phase 7: learned hierarchical search

- train global routing and local search values from accumulated outcomes;
- preserve transparent baseline features;
- scale by theorem and verified-lemma service;
- evaluate verified progress per unit cost.

16.7 First engineering sprint

A practical first sprint delivers:

1. version 3 schemas and examples;
2. FormalATPAdapter protocol and deterministic fake implementation;
3. formal soundness validator;
4. CLI demo that creates a formal run and replay result;
5. MORK atoms for declarations, states, tactic edges, alignments, and obstructions;
6. tests for all core invariants;
7. new OmegaClaw skill wrappers;
8. updated documentation and diagrams;
9. one end-to-end dry run from research claim to replay-verified formal result.

16.8 Architectural decision summary

Concern	Decision
Persistent agent	the proof project, not a conversation
Global semantic state	unified MORK-backed PeTTa metagraph
Durability	revisioned append-only event journal
Large content	content-addressed artifact store
Formal authority	exact Lean transition and independent replay
Strategic creativity	LLM and Hyperon outer loop
Local formal guidance	GNN tactic/premise policies plus PLN and search heuristics
Search topology	nested research and formal AND/OR graphs
Integration	bidirectional lemma and obstruction bridge
Mutation	deterministic commit gate only
Parallelism	project leases with fencing; run-local formal scheduling
Claim evaluation	discrete status plus feature vectors, not one scalar truth value
Formalization	available from the beginning and throughout the project

Appendix A. Unified MORK Atom Reference

A.1 Research search atoms

```
(proof p17 layer committed research-state rs101)
(proof p17 research-state-status rs101 open)
(proof p17 layer committed research-move rm204)
(proof p17 research-move-kind rm204 missing-lemma)
(proof p17 research-move-status rm204 queued)
(proof p17 out rs101 proposes rm204 re901)
(proof p17 in rm204 proposed-by rs101 re901)
```

A.2 Claim and alignment atoms

```
(proof p17 layer committed claim c42)
(proof p17 claim-status c42 provisional)
(proof p17 claim-statement-blob c42 b3_CLAIM)
```

```
(proof p17 layer committed formal-declaration fd17)
```

```
(proof p17 formal-declaration-name fd17 Proof17.clm42)
(proof p17 formal-source-blob fd17 b3_LEAN_SOURCE)
(proof p17 formal-environment fd17 env9)
```

```
(proof p17 layer committed alignment al9)
(proof p17 alignment-claim al9 c42)
(proof p17 alignment-declaration al9 fd17)
(proof p17 alignment-status al9 aligned)
(proof p17 alignment-review-blob al9 b3_REVIEW)
```

A.3 Formal run and state atoms

```
(proof p17 layer committed formal-run fr31)
(proof p17 formal-run-declaration fr31 fd17)
(proof p17 formal-run-status fr31 searching)
(proof p17 formal-run-environment fr31 env9)
(proof p17 formal-run-model-bundle fr31 mb7)
(proof p17 formal-root-state fr31 fs1)

(proof p17 layer committed formal-state fs1)
(proof p17 formal-state-run fs1 fr31)
(proof p17 formal-state-status fs1 open)
(proof p17 formal-state-exact-hash fs1 h_exact_1)
(proof p17 formal-state-semantic-signature fs1 h_sem_1)
(proof p17 formal-state-artifact fs1 art_state_1)
```

A.4 Tactic application atoms

```
(proof p17 layer committed tactic-application ta8)
(proof p17 tactic-source-state ta8 fs1)
(proof p17 tactic-name ta8 constructor)
(proof p17 tactic-arguments-blob ta8 b3_ARGS)
(proof p17 tactic-status ta8 pending)
(proof p17 out fs1 tactic-application ta8 fe44)
(proof p17 out ta8 requires fs2 fe45)
(proof p17 out ta8 requires fs3 fe46)
(proof p17 tactic-produced-subgoal-count ta8 2)
```

A.5 Heuristic annotations

```
(proof p17 gnn-tactic-prior ta8 score042)
(proof p17 premise-relevance ta8 score071)
(proof p17 pln-strength fs2 score038)
(proof p17 pln-confidence fs2 score081)
(proof p17 proof-number fs2 pn12)
(proof p17 scheduler-policy fr31 policy_gnn_pln_best_first_v1)
```

These annotations do not imply status transitions.

A.6 Certificates and replay

```
(proof p17 layer committed certificate cert5)
(proof p17 certificate-for-declaration cert5 fd17)
(proof p17 certificate-trace-blob cert5 b3_TRACE)
(proof p17 certificate-environment cert5 env9)
(proof p17 certificate-status cert5 candidate)

(proof p17 layer committed lean-replay lr6)
(proof p17 replay-certificate lr6 cert5)
(proof p17 replay-status lr6 verified)
(proof p17 replay-log-blob lr6 b3_LOG)
(proof p17 proved-by-formal-certificate c42 cert5)
```

A.7 Obstruction atoms

```
(proof p17 layer committed obstruction obs44)
(proof p17 obstruction-kind obs44 missing-lemma)
(proof p17 obstruction-run obs44 fr31)
(proof p17 obstruction-state obs44 fs88)
```

```
(proof p17 obstruction-artifact obs44 art_obs_44)
(proof p17 suggested-escalation obs44 invent-normalization-lemma)
```

A.8 Formal checkpoint event

```
{
  "event_id": "ev-00983",
  "proof_id": "p17",
  "expected_revision": 982,
  "new_revision": 983,
  "actor": "formal-atp-service",
  "lease": {"lease_id": "lease771", "fencing_token": 14},
  "operations": [
    {"op": "add", "atom": ["proof", "p17", "formal-run-status", "fr31", "searching"]},
    {"op": "add", "atom": ["proof", "p17", "formal-checkpoint", "fr31", "fcp12"]},
    {"op": "add", "atom": ["proof", "p17", "checkpoint-trace-blob", "fcp12", "b3_TRACE_CHUNK"]}
  ],
  "metadata": {
    "formal_sequence": 12,
    "environment_hash": "sha256:...",
    "frontier_digest": "sha256:...",
    "open_states": 41,
    "closed_states": 17
  }
}
```

Appendix B. Formal ATP and Worker Contracts

B.1 Formal-search request

The complete schema is supplied in `schemas/formal-search-request.schema.json`. It binds a run to proof, claim, declaration, alignment, lease, revision, environment, model, corpus, policy, and budget.

B.2 Formal-search result

The result includes:

- run identity and disposition;
- environment and model hashes;
- root and checkpoint state;
- all exported tactic edges with complete child lists;
- closed, dead, and open states;
- heuristic annotations;
- candidate certificate and replay status;
- verified sublemmas;
- obstructions;
- artifacts and budget use.

B.3 Tactic application

Every tactic application records:

- source state;
- tactic name and arguments;
- executor status;
- exact count and IDs of Lean-produced goals;
- diagnostic artifact on failure;

- model and policy provenance;
- child order when relevant;
- status derived from child closure.

B.4 Statement alignment

The alignment schema records the informal and formal objects, hashes, review checklist, differences, relation, and status.

B.5 Score vector

The score-vector schema contains independent named features. It has no truth field. A policy-specific `derived_priority` may be stored together with the policy ID and weights.

B.6 Worker result

Research workers retain the version 2 structured result discipline and add optional formal-target proposals, alignment requests, premise suggestions, and obstruction responses.

Appendix C. Prompt and Skill Specifications

C.1 Coordinator policy excerpt

The authoritative state is the proof workspace, not chat history.
 Use formal ATP directly when an aligned Lean declaration is available.
 Do not ask an LLM to imitate routine tactic search when the formal service can run it.
 Escalate formal stagnation through a typed obstruction, not a vague failure message.
 Never drop a Lean-produced subgoal.
 Never treat GNN, PLN, sampling, or LLM confidence as proof.
 Require independent Lean replay before formal completion.
 Never silently change theorem, assumptions, definitions, or quantifiers.
 Preserve all viable alternative research moves.
 Report durable milestones and semantic decisions, not every tactic expansion.

C.2 Formal ATP skill excerpt

Input: one exact formal-search request.
 Apply tactics only through the configured Lean/Pantograph executor.
 Record every successful tactic edge and every produced subgoal.
 Use heuristic scores only for scheduling.
 Checkpoint according to policy.
 Return a certificate only when the local root is closed.
 Request independent replay; do not self-certify.
 On failure or stagnation, emit a typed obstruction capsule.

C.3 Lemma bridge skill excerpt

Given a formal obstruction, propose the weakest useful bridge lemma and one or more cleaner variants. State exact prerequisites, relation to the current goal, first falsification test, and a Lean statement draft. Do not present the candidate as established. Preserve the source formal state and diagnostic IDs.

C.4 Statement-alignment skill excerpt

Compare the informal claim and Lean declaration field by field: quantifiers, domains, definitions, assumptions, classical principles, strength, and intended implication to the target. Return aligned, weaker, stronger, mismatch, or ambiguous with cited differences.

Appendix D. Configuration Example

```

version: 3
workspace_root: ./proof-workspace

store:
  semantic_backend: mork
  durability_backend: jsonl
  one_process_per_proof: true
  event_hash_chain: true
  checkpoint_every_events: 100
  maintenance_iteration_limit: 64

scheduler:
  global_policy: best_first_research_and_formal
  project_workers: 3
  lease_ttl_seconds: 1800
  formal_default_wall_seconds: 1800
  formal_stagnation_checkpoints: 3
  virtual_loss: 0.15

formal_atp:
  enabled: true
  adapter: mathematics_ai_atp
  transport: local_process
  retain_all_subgoals: true
  allow_heuristic_closure: false
  checkpoint_every_nodes: 250
  max_nodes: 10000
  max_depth: 64
  independent_replay: true
  deny_sorry: true
  accepted_axioms: []
  state_hash_version: 1
  semantic_transpositions: propose_only

models:
  tactic_policy_manifest: ./models/tactic-policy.json
  premise_policy_manifest: ./models/premise-policy.json
  pln_policy_manifest: ./models/pln-policy.json

context:
  research_token_budget: 60000
  obstruction_token_budget: 12000
  structural_weight: 4.0
  semantic_weight: 2.5
  pln_weight: 1.3
  include_divergent_strategy_when_stale: true

hyperon:
  enabled: true
  write_layer: speculative
  max_hypotheses_per_run: 12
  creative_operators:
    - missing-lemma-abduction
    - analogy-transfer
    - representation-change
    - obstruction-inversion
    - counterexample
    - premise-cluster

lean4:
  command: lake env lean
  project_path: ./lean
  pin_toolchain: true
  pin_mathlib: true

security:

```

```

allow_paths_under_workspace_only: true
worker_network_default: false
formal_network_default: false
shell_wall_seconds: 900
shell_memory_mb: 8192

```

References

1. iCog-Labs-Dev. *Mathematics-ai-atp*. GitHub repository, main branch, reviewed 16 July 2026.
2. iCog-Labs-Dev. `maths_ai/hybrid_reasoner/hypergraph.py` and `joint_inference.py`, reviewed 16 July 2026.
3. K. Yang et al. *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*. arXiv:2306.15626, 2023; LeanDojo project resources.
4. L. Aniva et al. *Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4*. arXiv:2410.16429; PyPantograph repository.
5. G. Lample et al. *HyperTree Proof Search for Neural Theorem Proving*. arXiv:2205.11491, 2022.
6. Lean Project. Lean 4 theorem prover and programming language, official documentation and repository.
7. Mathlib Community. `mathlib4` repository and documentation.
8. ASI Alliance. OmegaClaw-Core documentation and skill architecture.
9. TrueAGI. PeTTa repository and MORK-space documentation.
10. TrueAGI. MORK repository and wiki on data encoding, space operations, and MM2 transformations.
11. TrueAGI. MeTTa-native Probabilistic Logic Networks for Hyperon.
12. H. L. Greenblatt et al. ThreadKeeper architecture and bounded subagent documentation.
13. *A MORK-Native Multi-Agent Architecture for Long-Horizon Mathematical Theorem Proving*, version 2.0, companion package dated 15 July 2026.